Falcon LiDAR SDK | User Manual

Table of Contents

Table of Contents **1**
Preface **4**
1 Overview **5**
1.1 Introduction 5
1.2 Content of SDK Files 5
1.3 SDK Structure 6
1.3.1 LiDAR Module 6
1.3.2 Structure 6
1.4 Development Environment Requirements 7
2 SDK Usage **8**
2.1 SDK Operation Process 8
2.2 Data Processing of LiDAR 8
2.3 LiDAR Parameter Configuration 8
2.4 Get_PCD Sample 9
2.5 innovusion_lidar_util Use 9
3 SDK Reference **10**
3.1 Callback Function 10
3.1.1 InnoMessageCallback 10
3.1.2 InnoDataPacketCallback 10
3.1.3 InnoStatusPacketCallback 11
3.1.4 InnoLogCallback 12
3.2 API Reference 12
3.2.1 inno_api_version 12
3.2.2 inno_api_build_tag 12
3.2.3 inno_api_build_time 13
3.2.4 inno_lidar_setup_sig_handle 13
3.2.5 inno_lidar_set_logs 13
3.2.6 inno_lidar_set_log_level 14
3.2.7 inno_lidar_open_live 15
3.2.8 inno_lidar_open_file 15
3.2.9 inno_lidar_set_mode 16
3.2.10 inno_lidar_get_mode_status 17
3.2.11 inno_lidar_get_attribute 18
3.2.12 inno_lidar_get_attribute_string 18
3.2.13 inno_lidar_set_callbacks 19
3.2.14 inno_lidar_thread_setaffinity_np 20
3.2.15 inno_lidar_set_reflectance_mode 21
3.2.16 inno_lidar_set_return_mode 21
3.2.17 inno_lidar_set_roi 22

Copyright©Innovusion

3.2.18 inno_lidar_get_roi 22
3.2.19 inno_lidar_get_fw_state 23
3.2.20 inno_lidar_get_fw_version 23
3.2.21 inno_lidar_get_fw_sn 24
3.2.22 inno_lidar_get_model 25
3.2.23 inno_lidar_start 25
3.2.24 inno_lidar_stop 26
3.2.25 inno_lidar_close 26
3.3 Constants 27
3.3.1 Packet 27
3.3.2 Measurements 27
3.4 Enum 28
3.4.1 enum InnoLidarProtocol 28
3.4.2 enum InnoLidarState 29
3.4.3 enum InnoRecorderCallbackType 29
3.4.4 enum InnoLidarMode 30
3.4.5 enum InnoLidarStatus 31
3.4.6 enum InnoTimeSyncConfig 31
3.4.7 enum InnoTimeSyncType 32
3.4.8 enum InnoMessageLevel 33
3.4.9 enum InnoMessageCode 33
3.4.10 enum InnoFrameDirection 34
3.4.11 enum InnoItemType 35
3.4.12 enum InnoReflectanceMode 35
3.4.13 enum InnoConfidenceLevel 35
3.4.14 enum InnoMultipleReturnMode 36
3.4.15 enum InnoGalvoMode 36
3.4.16 enum InnoLogLevel 37
3.5 Structure 38
3.5.1 InnoBlockHeader 38
3.5.2 InnoChannelPoint 39
3.5.3 InnoBlock 40
3.5.4 InnoCommonHeader 40
3.5.5 InnoDataPacket 41
3.5.6 InnoStatusCounters 43
3.5.7 InnoStatusPacket 45
3.5.8 InnoStatusInFaults 46
3.5.9 InnoCommonVersion 49
3.5.10 InnoStatusSensorReadings 49
3.5.11 InnoMessage 50

Copyright©Innovusion

3.5.12 InnoXyzPoint 51
4 innovusion_lidar_util Reference 53
4.1 get_version 53
4.2 start <file_name> <capture_size_in_MB> 53
4.3 get_error_code 54
4.4 set_ip_address 54
4.5 set_network 54
4.6 get_network 55
4.7 get_uptime 55
4.8 get_framerate 55
4.9 soft_reboot 55
4.10 get_temperature 56
4.11 get_log 56
4.12 get_log_file 57
4.13 get_sn 57
4.14 get_status 57
4.15 get_model 58
4.16 get_config_changed 58

Preface

Product

inno-lidar-sdk-release-2.6.0

Manufacturer

INNOVUSION

Legal information

The information contained in this document is copyrighted by Innovusion, Inc. and is subject to change without notice. Innovusion endeavors to ensure the completeness and accuracy of the manual publications. When new or revised information becomes available, the entire document will be updated. Copyright 2022 Innovusion,Inc. All rights reserved.

Overview

inno-lidar-sdk-release-2.6.0 software development kit（hereinafter referred to as “SDK”）is specially designed for Innovusion Falcon LiDAR（hereinafter referred to as “Falcon” or “LiDAR”）. Users can access the LiDAR remotely and secondary development using the SDK. The content mainly introduces the usage of each function and the calling relationship between the functions.

The intended users of this manual include project developers (R&D personnel and designers) and service personnel.

Original document

This document is an original document of Innovusion.

Manual description

Although this document covers instructions to handle the frequent problems, it is still not guaranteed to get all problems fully solved. If you encounter other problems not covered in the manual, please contact Innovusion staff in time. This manual will be updated when new information becomes available.

Tel : (650)963-9573

E-mail: info@ innovusion.com

1 Overview

1.1 Introduction

The SDK is specially designed for Falcon, and it is developed based on C++ programming language. The SDK contains functions such as viewing/recording/replaying pointcloud files, log management, remote update, remote restarting/closing, parameter configuration (including system configuration, channel configuration, serial port configuration, alarm configuration, user configuration), etc.

1.2 Content of SDK Files

Table 1 SDK Files and Descriptions

Root	Secondary	Description
app	example	Sample and get_pcd executable file
	lidar_util	Configuration tool of the LiDAR parameter
	parse	LiDAR data parsing
	pcs	● inno_pc_server and inno_pc_client static executable files can replay pointcloud files ● inno_pc_npy.py script can transfer inno_pc_npy files into pcd files.
build	cpplint.py	Lint tools for C++ code analysis
lib		Static and dynamic library files
	libinnolidarsdk.so	SDK dynamic library
	libinnolidarsdkclient.so	Client SDK dynamic library. Recommended use.
	libinnolidarsdkcommon.so	Generic data processing interface and related tools
	libinnolidarutils.so	
	libinnolidarwsutils.so	
	libinnolidarsdk.a	SDK static library
	libinnolidarsdkclient.a	Client SDK static library. Recommended use.
	libinnolidarsdkcommon.a	Generic data processing interface and related tools
	libinnolidarutils.a	
	libinnolidarwsutils.a	
src	sdk_client	Compile to generate the library files
	sdk_common	API header files and structure header files, including inno_lidar_api.h and inno_lidar_packet.h.
	thirdparty	Opensource C++ library to parse JSON
	utils	Memory management, network communication management, thread management and file operation tools, etc
	ws_utils	Source files to generate ws_utils library files related to WebSocket operation

1.3 SDK Structure

1.3.1 LiDAR Module

Table 2 LiDAR module and description

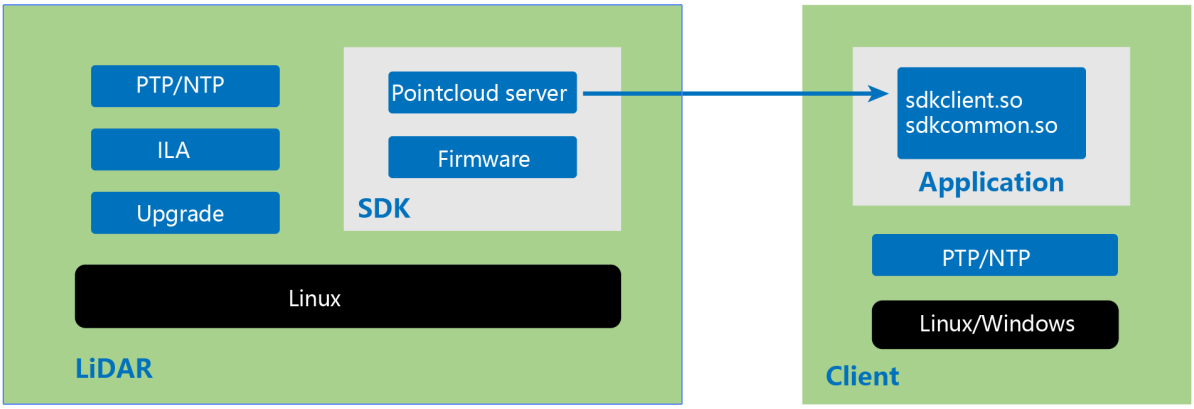
No.	Module	Description
1	Firmware	Initialization and control of peripheral hardware and related hardware interaction
2	Pointcloud Server	Forward/output data after reading
3	PTP/NTP	Time synchronization
4	ILA	LiDAR web management
5	Upgrade	LiDAR update web interface

1.3.2 Structure

You can use the SDK (Client or Server module) to obtain LiDAR data. The communication protocol can be TCP(LiDAR_IP:8010) and UDP broadcast (Client IP: Specified Receiving Port) when using Client SDK to read data. The communication protocol is TCP(LiDAR_IP:8002) when using SDK to read data. Client SDK+UDP is recommended.

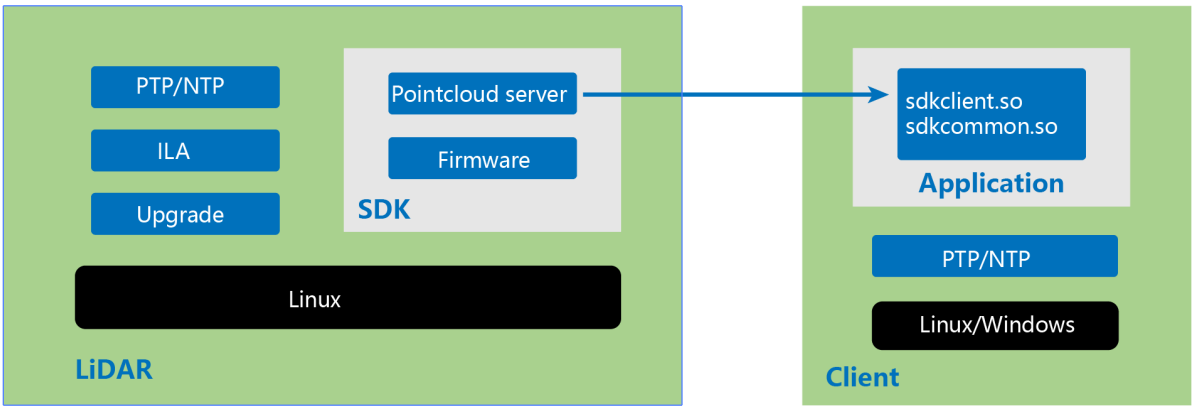
❖ **Using Client module to obtain LiDAR data**

The client load is small because it only undertakes the communication function, and small bandwidths are taken up in communication when using the Client SDK to obtain LiDAR data.



❖ **Using server module to obtain LiDAR data**

The client needs to additionally load inno_raw data parsing, noise filtering, format conversion and other functions. So extra bandwidth is taken up in communication when using the SDK to obtain LiDAR data.



Copyright©Innovusion

Caution	There will be issues if internal PCS is not turned off when using SDK to obtain LiDAR data. The steps to turn off the internal PCS. 1. Log into ILA. The default ILA login address is 172.168.1.10:8675. 2. Go to **Sensor config** > **Internal point cloud service.** 3. click **Stop** to turn off the internal PCS.

❖ Obtain data in pointcloud file

You can replay the pointcloud files using the SDK.

- Files in inno_pc/inno_pc_xyz format can be read using the Client SDK.
- Files in inno_raw format can be read using the SDK.

Caution	➤ You can use lidar_util tools to obtain pointcloud files in inno_raw format. ➤ You can use the get_pcd executable file to obtain pointcloud files in inno_pc/inno_pc_xyz format.

1.4 Development Environment Requirements

Table 3 Development Environment Requirements

Platform	Requirements
Hardware	X86_64/ARM64/MINGW64
Software	● Ubuntu 16.04 + gcc 5.4.0 or higher ● Ubuntu 18.04 + gcc 7.4.0 or higher ● Ubuntu 20.04 + gcc 7.5.0 or higher ● Windows 10+ 64Bit

2 SDK Usage

The SDK is developed based on C++ programming language. This chapter mainly introduces how to use the SDK and provides brief examples.

Caution	➤ Please notice that the integration with other languages of SDK is not supported by our company.

2.1 SDK Operation Process

The required steps to access the LiDAR with the SDK are given below. There are many optional function calls which are found in the inno_lidar_api.h header file.

1. open

   inno_lidar_open_file: Read pointcloud files in inno_raw/inno_pc/inno_pc_xyz format.

   inno_lidar_open_live: Read real-time LiDAR data. The open function's return value is the LiDAR handle. The LiDAR handle is necessary for all subsequent operations of LiDAR.

2. set callback

   inno_lidar_set_callbacks: Set callback function. Set callback function. You can get the callback function of the message, data and status.

   For the callback function related to the message, refer to **3.1.1 InnoMessageCallback**.

   For the callback function related to the data, refer to **3.1.2 InnoDataPacketCallback**.

   For the callback function related to the status, refer to **3.1.3 InnoStatusPacketCallback**.

   For other parameters and configurations, refer to **3.1 Callback Function** and **3.2 API Reference.**

3. start

   inno_lidar_start: Start reading data and calling the callback function. For details, refer to **3.1 Callback Function** and **3.5 Structure.**

4. stop

   inno_lidar_stop: Stop reading data.

5. close

   inno_lidar_close: Disconnect from LiDAR.

2.2 Data Processing of LiDAR

Each frame of LiDAR data contains multiple InnoDataPacket. Each InnoDataPacket consists of InnoCommonHeader, frame information, InnoBlock, etc.

- InnoCommonHeader contains information about the mode and status of the LiDAR. In addition to InnoDataPacket, InnoStatusPacket also includes InnoCommonHeader.
- For details of frame information, refer to **3.5.5 InnoDataPacket**.
- InnoBlock contains InnoBlockHeader and InnoChannelPoint. InnoBlockHeader contains information such as each point's angle. InnoChannelPoint contains information such as each point's distance and reflectivity.

Caution	You can view the declaration of InnoDataPacket in inno_lidar_packet.h.

2.3 LiDAR Parameter Configuration

You can set the parameters of LiDAR based on the usage purpose and scenes after obtaining the LiDAR handle. This section shows an example of LiDAR IP settings. For details, refer to **3 SDK Reference**.

Copyright©Innovusion

The default IP address of Falcon is 172.168.1.10. You can change the LiDAR IP through innovusion_lidar_util. For the usage of innovusion_lidar_util, refer to **4 innovusion_lidar_util Reference.**

LiDAR IP information will be broadcasted to LAN every 10 seconds in UDP. Users can capture the data packet through wireshark or tcpdump to retrieve the IP address.

The example of getting an IP address in tcpdump is as follows.

```
sudo tcpdump udp and host 255.255.255.255 -X
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eno1, link-type EN10MB (Ethernet), capture size 262144 bytes
17:58:37.864289 IP 10.42.0.91.49802 > 255.255.255.255.33333: UDP, length 48
        0x0000:  4500 004c b250 4000 4011 7dcc 0a2a 005b  E..L.P@.@.}..*.[
        0x0010:  ffff ffff c28a 8235 0038 8bbc 4950 3a31  .......5.8..IP:1
        0x0020:  302e 3432 2e30 2e39 3120 4e45 544d 4153  0.42.0.91.NETMAS
        0x0030:  4b3a 3235 352e 3235 352e 3235 352e 3020  K:255.255.255.0.
        0x0040:  4741 5445 5741 593a 6574 6830            GATEWAY:eth0
```

2.4 Get_PCD Sample

In this sample, the get_pcd executable file called the Client SDK interface to connect an external computer with the LiDAR and obtained the pointcloud data to record the data.

1. Unzip SDK tgz file to the inno_sdk folder.

```
mkdir -p inno_sdk
cd inno_sdk
tar xzvf ../inno-lidar-sdk-release-2.6.0.tgz
```

2. (Optional) You can view the README file. The README file contains a brief description of SDK content and structure.

```
less README
```

3. Recompile the executable file.

```
make
```

4. Run the get_pcd file.

```
cd apps/example
./get_pcd -h
```

5. Connect the LiDAR to obtain the pointcloud data.
   - Connect LiDAR with the power supply to start the LiDAR.
   - Connect LiDAR with the external computer. Change the computer IP to ensure the computer IP address and LiDAR IP address are in the same network segment. In this sample, the LiDAR IP address is 172.168.1.10.

```
./get_pcd --lidar-ip 172.168.1.10 --lidar-port 8010 --pcd-filename capture.pcd
pcl_viewer capture.pcd
```

2.5 innovusion_lidar_util Use

Users can change the configuration file, check the LiDAR status, obtain pointcloud data and do other functions through innovusion_lidar_util. For details, refer to **4 innovusion_lidar_util Reference.**

3 SDK Reference

- The declaration of the callback functions and API interfaces is in inno-sdk/src/sdk_common/inno_lidar_api.h.
- The definition of struct, enum and constant is in inno-sdk/src/sdk_common/inno_lidar_packet.h.

3.1 Callback Function

3.1.1 InnoMessageCallback

Obtain the operation information of LiDAR.

Syntax

```
/* @param lidar_handle The handle of the lidar that triggered the callback.
 * @param context Callback context passed in inno_lidar_set_callbacks().
 * @param from_remote > 0 means the message is from remote source
 * @param level Severity of message.
 * @param code Error code.
 * @param error_message Error message.
 * @return Void.
 */
typedef void (*InnoMessageCallback)(int lidar_handle,
                                    void *context,
                                    uint32_t from_remote,
                                    enum InnoMessageLevel level,
                                    enum InnoMessageCode code,
                                    const char *error_message);
```

Parameters

- **lidar_handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.
- **context**

  The context parameter passed in when the inno_lidar_set_callbacks function is called.
- **from_remote**

  When from_remote =1, the message is from the server.

  When from_remote = 0, the message is from the client.
- **level**

  The severity level of the message. For details, refer to 3.4.8 enum InnoMessageLevel.
- **code**

  The message code. It indicates the event type that generated the InnoMessageCallback. For details, refer to 3.4.9 enum InnoMessageCode.
- **error_message**

  The message is in string format.

Remarks

InnoMessageCallback is called when the events in InnoMessageCode occur.

3.1.2 InnoDataPacketCallback

SDK will transfer data to the client through the InnoDataPacketCallback function after receiving a complete InnoDataPacket data. The client will process the data in the callback function. For details, refer to 3.5.5 InnoDataPacket.

Copyright©Innovusion

Syntax

```
/*
* @param lidar_handle The handle of the lidar that triggered the callback.
* @param context Callback context passed in inno_lidar_set_callbacks().
* @param data Pointer to InnoDataPacket
* @return 0
*/
typedef int (*InnoDataPacketCallback)(int lidar_handle,
                                      void *context,
                                      const InnoDataPacket *data);
```

Parameters

- **lidar_handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **context**

  The context parameter passed in when the inno_lidar_set_callbacks function is called.

- **data**

  The InnoDataPacket pointer. For details, refer to 3.5.5 InnoDataPacket.

Return values

If the return value is not equal to 0, the data transmission of LiDAR will be interrupted, and the SDK will be restarted.

3.1.3 InnoStatusPacketCallback

SDK will notify the client through InnoStatusPacketCallback function after receiving the status information from the LiDAR.

Syntax

```
/*
   * @param lidar_handle The handle of the lidar that triggered the callback.
   * @param context Callback context passed in inno_lidar_set_callbacks().
   * @param status Pointer to InnoDataPacket
   * @return 0
   */
  typedef int (*InnoStatusPacketCallback)(int lidar_handle,
                                          void *context,
                                          const InnoStatusPacket *status);
```

Parameters

- **lidar_handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **context**

  The context parameter passed in when the inno_lidar_set_callbacks function is called.

- **status**

  The InnoDataPacket pointer. Used to Indicate the LiDAR status.

Return values

The return value is always zero.

Remarks

The LiDAR sends InnoStatusPacket every 50 ms.

3.1.4 InnoLogCallback

SDK will notify the log to the client by InnoStatusPacketCallback after receiving the key log from the LiDAR. The client can record the log locally to facilitate subsequent troubleshooting.

Syntax

```
/*
 *  @param callback context
 *  @param level Log level.
 *  @param header1 First log message header, e.g.
 *         '2018-12-06 23:54:09'
 *  @param header2 Second log message header, e.g.
 *       '[ERROR] 7252 rawdata.cpp:3259'
 *  @param msg Log message body.
 *  @return Void.
 */
typedef void (*InnoLogCallback)(void *ctx
                                enum InnoLogLevel level
                                const char *header1
                                const char *header2
                                const char *msg);
```

Parameters

- **ctx**

  The context parameter passed in when the inno_lidar_set_callbacks function is called.
- **level**

  The log level. For details, refer to 3.4.16 enum InnoLogLevel.
- **header1**

  The time information in the log. Format sample: ‘2018-12-06 23:54:09’.
- **header2**

  The location information in the log. Format sample:’[ERROR] 7252 rawdata.cpp:3259’.
- **msg**

  The details of the log. When msg="message info", inno_log_info("message info") function is called.

3.2 API Reference

3.2.1 inno_api_version

Obtain the version of SDK API to verify if the component version matches.

Syntax

```
/*
 * @brief Get Innovusion lidar API version
 * @return Version string.
 */
const char *inno_api_version(void);
```

Return values

SDK API version information.

3.2.2 inno_api_build_tag

Obtain the build tag of SDK API to verify if the components’ build tag matches.

Copyright©Innovusion

Syntax

```
/*
 * @brief Get Innovusion lidar API build tag.
 * @return Build tag string.
 */
const char *inno_api_build_tag(void);
```

Return values

SDK build tag.

3.2.3 inno_api_build_time

Obtain the build time of SDK API to verify if the components' build time matches.

Syntax

```
/*
 * @brief Get Innovusion lidar API build time.
 * @return Build time string.
 */
const char *inno_api_build_time(void);
```

Return values

SDK build time.

3.2.4 inno_lidar_setup_sig_handle

Stack backtrace will be printed and the previous signal handler will be called when a SIGSEGV, SIGBUS, or SIGFPE error occurs in the LiDAR.

Syntax

```
/*
 * @brief Setup sigaction for SIGSEGV SIGBUS SIGFPE (optional)
 *        The signal handler will print out the stack backtrace
 *        and then call old signal handler.
 * @return Void.
 */
void inno_lidar_setup_sig_handler();
```

3.2.5 inno_lidar_set_logs

Set the log parameters and callback functions.

Syntax

```
/*
 * @brief Set log files fds (file descriptors).
 *        This function can be only called once.
 * @param out_fd All log whose level is less or equals
 *        to INNO_LOG_LEVEL_INFO will be written to this file.
 *        -1 means do not write.
 * @param error_fd All log whose level is bigger or equals
 *        to INNO_LOG_LEVEL_WARNING will be written to this file.
 *        -1 means do not write.
 * @param rotate_file_base_file
 * @param rotate_file_number
 * @param rotate_file_size_limit
 * @param log_callback
 * @param callback_ctx
 * @param rotate_file_base_file_err
 * @param rotate_file_number_err
 * @param rotate_file_size_limit_err
 * @return Void.
 */
void inno_lidar_set_logs int out_fd
                         int error_fd
                         const char *rotate_file_base_file
```

```
                                uint32_t rotate_file_number,
                                uint64_t rotate_file_size_limit,
                                InnoLogCallback log_callback,
                                void *callback_ctx,
                                const char *rotate_file_base_file_err = NULL,
                                uint32_t rotate_file_number_err = 0,
                                uint64_t rotate_file_size_limit_err = 0);
```

Parameters

- **out_fd**

  The standard output file descriptors. When out_fd=1, the standard output can be disabled.
- **error_fd**

  The error output file descriptors. When error_fd=1, the error output can be disabled.
- **rotate_file_base_file**

  The log file path for recording the standard output.
- **rotate_file_number**

  The number of log files for recording the standard output.
- **rotate_file_size_limit**

  The maximum size of the single log file for recording the standard output.
- **log_callback**

  The log callback function. For details, refer to **3.1.4 InnoLogCallback.**
- **callback_ctx**

  The context corresponds to the callback function.
- **rotate_file_base_file_err**

  The log file path for recording the error output.
- **rotate_file_number_err**

  The number of log files for recording the error output.
- **rotate_file_size_limit_err**

  The maximum size of the single log file for recording the error output.

Remarks

- The log with a level less than or equal to INNO_LOG_LEVEL_WARNING will be outputted as an error. For details, refer to **3.4.16 enum InnoLogLevel.**
- rotate_file_base_file_err cannot be set separately. rotate_file_base_file should be set simultaneously. rotate_file_base_file can be set independently.

3.2.6 inno_lidar_set_log_level

Set the log level. If the log level is higher than the setting value, the log will not be output.

Syntax

```
    /*
   * @brief Set debug level for all lidar handles.
   *        This function can be called any time.
   *        It may be called multiple times, e.g. to decrease the log
   *        level before executing some code and then increasing the
   *        log level again after the section of code completes.
   * @param log_level Set new debug level.
   * @return Void.
   */
  void inno_lidar_set_log_level(enum InnoLogLevel log_level);
```

Parameters

- **log_level**

Copyright©Innovusion

The SDK log level. For details, refer to 3.4.16 enum InnoLogLevel

3.2.7 inno_lidar_open_live

Obtain the LiDAR handle after the LiDAR has been connected.

Syntax

```
/*
 * @brief Open a lidar handle for a live Innovusion lidar unit.
 * @param name Contains the name of lidar, e.g. "forward-1"
 *        The name should be less than 32 characters.
 * @param lidar_ip Contains the lidar IP string, e.g. "172.168.1.10".
 * @param port Lidar PORT, e.g. 8001.
 * @param protocol Specify protocol,
 * @param udp_port UDP port to receive data, when
 *                 protocol is INNO_LIDAR_PROTOCOL_UDP
 * @return Return lidar handle
 */
int inno_lidar_open_live(const char *name
                         const char *lidar_ip
                         uint16_t port
                         enum InnoLidarProtocol protocol
                         uint16_t udp_port);
```

Parameters

- **name**

  The LiDAR name. There are no specific requirements for the identifiers.

- **lidar_ip**

  The LiDAR IP address.

- **port**

  The port name. The port is used by the client to communicate with LiDAR via TCP.

- **protocol**

  The protocol. The protocol is used by the client to communicate the LiDAR. For details, refer to 3.4.1 enum InnoLidarProtocol.

- **udp_port**

  The UDP port name. The UDP port is used by the client to communicate with LiDAR. The system will assign one port randomly by default.

Return values

The LiDAR handle. Other API operations need this value to locate the corresponding LiDAR.

3.2.8 inno_lidar_open_file

Obtain the pointcloud files. Users can obtain the files in inno_pc and inno_pc_xyz format via Client SDK. Users can obtain the files in inno_raw format via SDK.

Syntax

```
/*
 * @brief Open a lidar handle from an Innovusion lidar proprietary
 *        data file
 * @param name Name of lidar.
 *        The name should be less than 32 characters.
 * @param filename The filename of an Innovusion lidar
 *        proprietary data file.
 * @param raw_format
 * @param play_rate The playback rate,
 *        == 0 means play as fast as possible
 *        <= 100 means MB/s, e.g. 15 means play at 15 MB/s
 *         > 100 means rate_multiplier = play_rate / 10000.0,
 *           e.g, 15000 means play at 1.5x speed
 * @param rewind How many times rewind before stop, 0 means no rewind,
```

```
     *        < 0 means rewind infinity times.
     * @return Return a lidar handle.
     */
    int inno_lidar_open_file(const char *name
                             const char *filename
                             bool raw_format
                             int play_rate
                             int rewind
                             int64_t skip);
```

Parameters

- **name**

  The LiDAR name. There are no specific requirements for the identifiers.

- **filename**

  The input file.

- **raw_format**

  When raw_format=true, the raw data can be used directly for the InnoDataPacket callback.

  When raw_format=false, the processed data is used for the InnoDataPacket callback.

- **play_rate**

  When the play_rate=0, the play speed is as fast as possible.

  When the play_rate≤100, the unit of play speed is mb/s.

  When the play_rate>100, the play speed=play_rate/10000.0. e.g., When play_rate=15000. the play speed is at 1.5 times.

- **rewind**

  The number of times to read the file. rewind=0 means no repeat. A negative rewind value means the file will be repeated indefinitely until the program is manually stopped.

- **skip**

  Frames skipped.

Return values

The LiDAR handle. Other API operations need this value to locate the corresponding LiDAR.

Remarks

There are several ways to record pointcloud files.

- You can record and download pointcloud files via ILA.
- You can record pointcloud files in inno_pc, inno_pc_xyz, pcd, csv, png, and bag format via the get_pcd execute file.
- You can record pointcloud files in inno_pc, raw, png, and bag format via inno_pc_client or inno_pc_server.
- You can record pointcloud files in inno_raw format via innovusion_lidar_util.

3.2.9 inno_lidar_set_mode

Set the working mode （e.g., sleep mode, short range mode, normal mode, etc.）of the LiDAR.

Syntax

```
    /*
     * @brief Set lidar working mode
     * @param mode Target mode
     * @param mode_before_change Mode before this change
     * @param status_before_change Lidar Status before this change
     * @return 0 means success, otherwise failure (e.g., invalid lidar handle)
     */
    int inno_lidar_set_mode(int handle
```


Copyright©Innovusion

```
                                enum InnoLidarMode mode
                                enum InnoLidarMode *mode_before_change
                                enum InnoLidarStatus *status);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **mode**

  The working mode of the LiDAR. For details, refer to 3.4.4 enum InnoLidarMode.

- **mode_before_change**

  The pointer passes this parameter to obtain the previous LiDAR working mode. For details, refer to 3.4.4 enum InnoLidarMode.

- **status**

  The pointer passes the parameter to obtain the LiDAR status. For details, refer to 3.4.5 enum InnoLidarStatus.

Return values

A return value of 0 means successful configuration. Other return values mean configuration failed.

3.2.10 inno_lidar_get_mode_status

Obtain the working mode of the LiDAR.

Syntax

```
/*
 * @brief Get lidar current working mode and status
 * @param handle Lidar handle
 * @param mode Current work mode
 * @param pre_mode Previous work mode
 * @param status Work status
 * @param in_transition_mode_ms Time (in ms) stay in transition mode
 * @return 0 means success, otherwise failure (e.g. invalid lidar handle)
 */
int inno_lidar_get_mode_status(int handle
                               enum InnoLidarMode *mode
                               enum InnoLidarMode *pre_mode
                               enum InnoLidarStatus *status
                               uint64_t *in_transition_mode_ms);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **mode**

  The pointer passes this parameter to obtain the current working mode of the LiDAR. For details, refer to 3.4.4 enum InnoLidarMode.

- **pre_mode**

  The pointer passes this parameter to obtain the previous LiDAR working mode. For details, refer to 3.4.4 enum InnoLidarMode.

- **status**

  The pointer passes this parameter to obtain the LiDAR status. For details, refer to 3.4.5 enum InnoLidarStatus.

- **in_transition_mode_ms**

Time to switch between the working modes.

Return values

A return value of 0 means successful configuration. Other return values mean configuration failed.

3.2.11 inno_lidar_get_attribute

Obtain the attribute value of the double data type in LiDAR. The specific value of attributes needs to be combined with customer requirements.

Syntax

```
/*
 * @brief Get lidar attribute like frame_rate
 * @param handle Lidar handle
 * @param attribute Name of the attribute
 * @param value Store the value of the attribute
 * @return 0 means success, otherwise failure (e.g., invalid lidar handle)
 */
int inno_lidar_get_attribute(int handle
                             const char *attribute
                             double *value);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **attribute**

  The attribute name, such as frame_rate, reflection_mode, return_mode, etc.

- **value**

  The pointer passes this parameter to obtain the attribute value.

Return values

A return value of 0 means successful configuration. Other return values mean configuration failed.

3.2.12 inno_lidar_get_attribute_string

Obtain the attribute value of the string data type in LiDAR. The specific value of attributes needs to be combined with customer requirements.

Syntax

```
/*
 * @brief Get lidar attribute like frame_rate
 * @param handle Lidar handle
 * @param attribute Name of the attribute
 * @param buffer Buffer to store the string value
 * @param Buffer_len Length of buffer, the recommended buffer_len is
 *                 1024, the size of buffer needs to be >= buffer_len.
 * @return 0 means success, otherwise failure (e.g., invalid lidar handle)
 */
int inno_lidar_get_attribute_string(int handle
                                    const char *attribute
                                    char *buffer  size_t buffer_len);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file

- **attribute**

  The attribute name, such as source_is_live_lidar, DID_ECU_MFG_DATE, DID_ECU_SN, etc.

- **buffer**

Copyright©Innovusion

The address of the buffer for storing the attribute value.

- **buffer_len**

  The size of the buffer for storing the attribute value.

Return values

A return value of 0 means successful configuration. Other return values mean configuration failed.

3.2.13 inno_lidar_set_callbacks

Set the callback function of LiDAR and the corresponding context.

Syntax

```
    /*
     * @brief Set callbacks for a lidar handle. Developers need to
     *        set callbacks before calling inno_lidar_start().
     *        For every lidar handle, inno_lidar_set_callbacks can
     *        only be called once.
     * @param handle Lidar handle (from either inno_lidar_open_live or
     *        inno_lidar_open_file).
     * @param message_callback This callback is called when some warning/error
happens.
     *                          For a given lidar handle, no message_callback or
     *                          frame_callback will be called until the
     *                          previous callback (for the same lidar handle)
     *                          has returned.
     *                          Multiple callbacks may happen at the same time
     *                          for different lidar handles.
     *                          Pass NULL pointer means no callback.
     * @param data_callback This callback is called when one frame or
     *                          sub-frame is available.  The callback happens
     *                          in a thread dedicated to that lidar_handle.
     *                          For a given lidar handle, no message_callback or
     *                          frame_callback will be called until the
     *                          previous callback (for the same lidar handle)
     *                          has returned.
     *                          Multiple callbacks may happen at the same time
     *                          for different lidar handles.
     *                          Pass NULL pointer means no callback.
     * @param status_callback
     * @param get_host_time This callback will get the current host time.
     *                         Pass NULL pointer means
clock_gettime(CLOCK_REALTIME)
     *                         will be used.
     * @param callback_context Context passed in when callback is invoked.
     * @return 0 means success, otherwise failure (e.g., invalid lidar handle)
     */
    int inno_lidar_set_callbacks(int handle,
                                 InnoMessageCallback message_callback,
                                 InnoDataPacketCallback data_callback,
                                 InnoStatusPacketCallback status_callback,
                                 InnoHosttimeCallback get_host_time,
                                 void *callback_context);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **message_callback**

  InnoMessageCallback. The function will be called when some warnings or errors occur. The next message_callback and data_callback procedures cannot be started until the callback procedure is completed. Please notice that different LiDARs can call the same InnoMessageCallback simultaneously.

- **data_callback**

InnoDataPacketCallback. The function will be called when a frame or a subframe data is valid. The next message_callback and data_callback cannot be started until the callback procedure is completed. Please notice that different LiDARs can call the same InnoMessageCallback simultaneously.

- **status_callback**

  InnoStatusPacketCallback. The function will be called when the LiDAR reports status actively.

- **get_host_time**

  InnoHosttimeCallback. The function will be called when the current host time is obtained. If the value of get_host_time is NULL, clock_gettime(CLOCK_REALTIME) will be used to obtain the current host time.

- **callback_context**

  The corresponding context of the callback function. It will be set to the client as the parameters of the callback function.

Return values

A return value of 0 means successful configuration. Other return values mean configuration failed.

3.2.14 inno_lidar_thread_setaffinity_np

Specify the CPU affinity related to LiDAR threads. This function only can be called before inno_lidar_start. For details, refer to pthread_setaffinity_np function from the c system library <pthread.h> (Websites: https://man7.org/linux/man-pages/man3/pthread_setaffinity_np.3.html).

Syntax

```
/*
 * @brief Specify the CPU affinity for all threads created by the
 *        library for this lidar instance. This function can only
 *        be called before inno_lidar_start() is called. Please see
 *        pthread_setaffinity_np() for reference.
 * @param handle Lidar handle.
 * @param cpusetsize size of *cpuset.
 * @param cpuset cupset that will be passed to pthread_setaffinity_np().
 * @param exclude_callback_thread 1 means do not set affinity for callback
 *        thread, 0 otherwise.
 * @return 0 means successful stored the affinity settings that will be
 *         passed in when calling pthread_setaffinity_np(). It doesn't
 *         not guarantee that pthread_setaffinity_np() will be successful.
 *         1 invalid handle
 */
int inno_lidar_thread_setaffinity_np(int handle
                                     size_t cpusetsize
                                     const cpu_set_t *cpuset
                                     int exclude_callback_thread);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **cpusetsize**

  The size of cpuset buffer.

- **cpuset**

  The affinity of threads to different CPUs. cpuset is the parameter transferred to pthread_setaffinity_np.

- **exclude_callback_thread**

Copyright©Innovusion

When exclude_callback_thread=1, do not set the affinity for the callback thread.

When exclude_callback_thread=0, set the affinity for the callback thread.

Return values

A return value of 0 means storing the affinity is successful. The affinity is only passed in when pthread_setaffinity_np is called, however this does not indicate that pthread_setaffinity_np has been set successfully.

A return value of 1 means the LiDAR handle is invalid.

Remarks

- The function should be called before inno_lidar_start.
- pthread_setaffinity_np is the system library function in pthread.h which is used to set a thread's affinity to the CPU. For details, refer to https://man7.org/linux/man-pages/man3/pthread_setaffinity_np.3.html.

3.2.15 inno_lidar_set_reflectance_mode

Set the LiDAR reflectance mode.

Syntax

```
/*
 * @brief Change the meaning of the ref data in InnoChannelPoint
 *        structures to indicate either reflectance or intensity.
 * @param handle Lidar handle
 * @param mode Either INNO_REFLECTANCE_MODE_INTENSITY
 *        or INNO_REFLECTANCE_MODE_REFLECTIVITY
 *        default is INNO_REFLECTANCE_MODE_REFLECTIVITY
 *        since 1.5.0
 * @return 0 means success, otherwise failure (e.g. invalid handle).
 */
int inno_lidar_set_reflectance_mode(int handle
                                    enum InnoReflectanceMode mode);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **mode**

  The LiDAR reflectance mode. For details, refer to **3.4.12 enum InnoReflectanceMode**.

Return values

A return value of 0 means successful configuration. Other return values mean configuration failed.

3.2.16 inno_lidar_set_return_mode

The return mode after a single laser emission of the LiDAR.

Syntax

```
/*
 * @brief Change the number & type of points returned each laser pulse.
 * @param handle Lidar handle.
 * @param ret_mode
 * @return 0 means success, otherwise failure (e.g. invalid handle).
 */
int inno_lidar_set_return_mode(int handle
                               enum InnoMultipleReturnMode ret_mode);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **ret_mode**

The return mode after a single laser emission of the LiDAR. For details, refer to 3.4.14 enum InnoMultipleReturnMode.

Return values

A return value of 0 means successful configuration. Other return values mean configuration failed.

3.2.17 inno_lidar_set_roi

Set the center position of ROI.

Syntax

```
/*
 * @brief Set ROI (center point)
 * @param handle Lidar handle.
 * @param horz_angle ROI horizontal center (in [-60, 60] degrees)
 * @param vert_angle ROI vertical center (in [-25, 25] degrees)
 *        Setting either horz_angle or vert_angle to
 *        kInnoNopROI, i.e. 10000.0
 *        will maintain the previous value for that angle.  Any other value
 *        outside the allowed range will result in a failure code return and
 *        no change in ROI center.
 * @return 0 means success, otherwise failure (e.g. invalid handle)
 */
int inno_lidar_set_roi(int handle  double horz_angle  double vert_angle);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **horz_angle**

  The horizontal angle of the ROI center. The range is from -60 degrees to 60 degrees.

- **vert_angle**

  The vertical angle of the ROI center. The range is from -25 degrees to 25 degrees.

Return values

A return value of 0 means successful configuration. Other return values mean configuration failed.

Remarks

- Any values beyond the range cannot be set successfully, and the ROI center position will not be changed.
- The ROI range is fixed. You can only set the center position of ROI.
- The ROI described in the parameters is for standard Falcon configuration. Customers may have narrower range due to customization for their specific use case.

3.2.18 inno_lidar_get_roi

Get the center position of ROI.

Syntax

```
/*
 * @brief Get current ROI (center point)
 * @param handle Lidar handle.
 * @param horz_angle Store return ROI horizontal center (in [-60, 60] degrees)
 * @param vert_angle StoreROI vertical center (in [-25, 25] degree)
 * @return 0 means success, otherwise failure (e.g. invalid handle)
 */
int inno_lidar_get_roi(int handle  double *horz_angle  double *vert_angle);
```

Parameters

- **handle**

Copyright©Innovusion

The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **horz_angle**

  The horizontal angle of the ROI center. The range is from -60 degrees to 60 degrees.

- **vert_angle**

  The vertical angle of the ROI center. The range is from -25 degrees to 25 degrees.

Return values

A return value of 0 means successful configuration. Other return values mean configuration failed.

3.2.19 inno_lidar_get_fw_state

Get the state of the LiDAR.

Syntax

```
/*
 * @brief Description: Query lidar unit's state
 * @param handle Lidar handle.
 * @param state
 * @param error_code Address to store error code. NULL means
 *        do not store error code
 * @return 0 means success, -1 if handle is invalid
 */
int  inno_lidar_get_fw_state(int handle
                             InnoLidarState *state
                             unsigned int *error_code);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **state**

  The pointer passes this parameter to obtain the LiDAR status.

- **error_code**

  The address for storing the error code. The error_code is NULL means there is no error code.

Return values

A return value of 0 means successful configuration. A return value of -1 means the LiDAR handle is invalid.

3.2.20 inno_lidar_get_fw_version

Get the firmware version of the LiDAR.

Syntax

```
/*
 * @brief Query lidar unit's firmware version.
 * @param handle Lidar handle.
 * @param buffer Buffer to store the firmware version.
 * @param Buffer_len Length of buffer, the recommended buffer_len is
 *                  512, the size of buffer needs to be >= buffer_len.
 * @return 0 cannot get firmware version
 *        -1 invlid lidar handle
 *        -2 buffer_len is too small
 *        -3 source is file
 *        otherwise return the size of firmware version string
 *         stored in buffer, not include the trailing zero
 *        Sample content in the buffer:
 *           App Version: app-2.3.0-rc8.134
 *             build-time: 2019-08-14-18-19-25
 *           FPGA Datecode: 0x190814e2
 *             fpga-ver: 0x13
```

```
 *              fpga-rev: 0x0e
 *              board-rev: 0x2
 *            Firmware Version: 2.3.1-rc3-418.2019-08-15-17-41-40
 *              build-tag: 2.3.1-rc3-418
 *              build-time: 2019-08-15-17-41-40
 *              build-git-tag: 1.0.19
 */
int inno_lidar_get_fw_version(int handle  char *buffer  int buffer_len);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **buffer**

  The buffer for storing the LiDAR firmware version.

- **buffer_len**

  The size of the buffer where the LiDAR firmware version is stored. 512 is recommended. The actual size of the buffer should be exceeded buffer_len.

Return values

- A return value of 1 means that the LiDAR firmware version cannot be obtained.
- A return value of -1 means the LiDAR handle is invalid.
- A return value of -2 means the buffer_len is too small.
- A return value of -3 means the data source is the pointcloud file.
- Other return values indicate the size of buffer for storing the message. (The size does not include 0 at the end of the string)

3.2.21 inno_lidar_get_fw_sn

Get the serial number of the LiDAR.

Syntax

```
/*
 * @brief Query lidar unit's S/N
 * @param handle Lidar handle.
 * @param buffer Buffer to store the S/N.
 * @param buffer_len Length of buffer, the recommended buffer_len is
 *                 128, the size of buffer needs to be >= buffer_len.
 * @return 0 cannot get S/N
 *         -1 invlid lidar handle
 *         -2 buffer_len is too small
 *         -3 source is file
 *         otherwise return the size of S/N string
 *         stored in buffer, not include the trailing zero
 */
int inno_lidar_get_sn(int handle, char *buffer, int buffer_len);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **buffer**

  The buffer where the serial number of the LiDAR is stored.

- **buffer_len**

  The size of the buffer where the LiDAR serial number is stored. 128 is recommended. The actual size of the buffer should be exceeded buffer_len.

Return values

- A return value of 0 means that the LiDAR serial number cannot be obtained.
- A return value of -1 means the LiDAR handle is invalid.

Copyright©Innovusion

- A return value of -2 means the buffer_len is too small.
- A return value of -3 means the data source is the pointcloud file.
- Other return values indicate the buffer size used to store the message. (The size does not include 0 at the end of the string)

3.2.22 inno_lidar_get_model

Get the LiDAR model.

Syntax

```
/*
 * @brief Query lidar unit's model
 * @param handle Lidar handle.
 * @param buffer Buffer to store the model.
 * @param buffer_len Length of buffer, the recommended buffer_len is
 *        32, the size of buffer needs to be >= buffer_len
 * @return 0 cannot get model
 *        -1 invlid lidar handle
 *        -2 buffer_len is too small
 *        -3 source is file
 *        otherwise return the size of model string
 *        stored in buffer, not include the trailing zero
 */
int inno_lidar_get_model(int handle, char *buffer, int buffer_len);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

- **buffer**

  The buffer for storing the message.

- **buffer_len**

  The size of the buffer for storing the message. 32 is recommended. The actual size of buffer should be exceeded buffer_len.

Return values

- A return value of 0 means that the LiDAR model cannot be obtained.
- A return value of -1 means the LiDAR handle is invalid.
- A return value of -2 means the buffer_len is too small.
- A return value of -3 means the data source is the pointcloud file.
- Other return values indicate the size of the buffer for storing the message. (The size does not include 0 at the end of the string)

3.2.23 inno_lidar_start

Start to receive the data.

- If inno_api_open_live is called after inno_lidar_start, read the data sent by the LiDAR.
- If inno_api_open_life is called after inno_lidar_start, read the data in the pointcloud file.

Syntax

```
/*
 * @brief Start to read data from live lidar unit or from a data file.
 *        The message_callback, frame_callback and cframe_callback will
 *        be called only after this function is called.  They may
 *        be called before this function returns since they are in
 *        different threads.
 * @param handle Lidar handle.
 * @return 0 means success, otherwise failure (e.g. invalid handle).
 */
int inno_lidar_start(int handle);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

Return values

A return value of 0 means successful configuration. A return value of 1 means the LiDAR handle is invalid.

Remarks

- message_callback, frame_callback, and cframe_callback can only be called after the inno_lidar_start.
- inno_lidar_open_live may be called before inno_lidar_start is returned because they run in different threads.

3.2.24 inno_lidar_stop

Stop reading the data from the LiDAR or the file.

Syntax

```
/*
 * @brief Stop reading data from live lidar unit or from a data file.
 *        The message_callback, frame_callback and cframe_callback will
 *        not be called once this function is returned.
 * @param handle Lidar handle.
 * @return 0 means success, otherwise failure (e.g. invalid handle)
 */
int inno_lidar_stop(int handle);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

Return values

A return value of 0 means successful configuration. A return value of 1 means the LiDAR handle is invalid.

Remarks

message_callback, frame_callback, and cframe_callback will not be called after inno_lidar_stop returned.

3.2.25 inno_lidar_close

Disconnect from the LiDAR and release all resources.

Syntax

```
/*
 * @brief Close the lidar handle and release all resources.
 *        This function can be called on a valid lidar handle only
 *        after inno_lidar_stop() call has returned,
 *        or inno_lidar_start() has never been called before.
 * @param handle Lidar handle.
 * @return 0 means success, otherwise failure (e.g. invalid handle).
 */
int inno_lidar_close(int handle);
```

Parameters

- **handle**

  The LiDAR handle. Created via inno_api_open_live or inno_api_open_file.

Return values

A return value of 0 means successful configuration. A return value of 1 means the LiDAR handle is invalid.

Remarks

- inno_lidar_close only can be called after inno_lidar_stop.

Copyright©Innovusion

- It is recommended to confirm that the LiDAR handle is valid before calling inno_lidar_close.

3.3 Constants

3.3.1 Packet

Syntax

```
static const uint16_t kInnoMagicNumberDataPacket = 0x176A;
static const uint8_t kInnoMajorVersionDataPacket = 1;
static const uint8_t kInnoMinorVersionDataPacket = 2;
static const uint16_t kInnoMagicNumberStatusPacket = 0x186B;
static const uint8_t kInnoMajorVersionStatusPacket = 1;
static const uint8_t kInnoMinorVersionStatusPacket = 2;
```

Parameters

- **kInnoMagicNumberDataPacket**=0x176A

  The Magic number of DataPacket. Used to verify the correctness of the DataPacket.
- **kInnoMajorVersionDataPacket**=1

  The major version of the DataPacket.
- **kInnoMinorVersionDataPacket=**2

  The minor version of the DataPacket.
- **kInnoMagicNumberStatusPacket**=0x186B

  The Magic number of StatusPacket. Used to verify the correctness of the StatusPacket.
- **kInnoMajorVersionStatusPacket**=1

  The major version of the StatusPacket.
- **kInnoMinorVersionStatusPacket**=2

  The minor version of the StatusPacket.

3.3.2 Measurements

Syntax

```
static const uint32_t kInnoDistanceUnitPerMeter = 200;
static const double kMeterPerInnoDistanceUnit =    1.0 /
kInnoDistanceUnitPerMeter;
static const uint32_t kInnoDegreePerPiRad = 180;
static const uint32_t kInnoAngleUnitPerPiRad = 32768;
static const double kRadPerInnoAngleUnit = M_PI / kInnoAngleUnitPerPiRad;
static const double kDegreePerInnoAngleUnit =    180.0 / kInnoAngleUnitPerPiRad;
static const double kInnoAngleUnitPerDegree = kInnoAngleUnitPerPiRad / 180.0;
static const uint32_t kInnoChannelNumberBit = 2;
static const uint32_t kInnoChannelNumber = 1 << kInnoChannelNumberBit;
static const uint32_t kInnoMaxMultiReturn = 2;
static const int16_t kInnoVAngleDiffBase = 196;

static const double kInnoNopROI = 10000.0;
```

Parameters

- **kInnoDistanceUnitPerMeter**=200

  The number of minimum distance units per meter.
- **kMeterPerInnoDistanceUnit**=1.0/kInnoDistanceUnitPerMeter

  The unit of minimum distance.
- **kInnoDegreePerPiRad**=180

  The degrees per PI.
- **kInnoAngleUnitPerPiRad**=32768

  The number of minimum degree units per PI.

- **kRadPerInnoAngleUnit**= M_PI/kInnoAngleUnitPerPiRad
  The minimum unit in radius.
- **kDegreePerInnoAngleUnit**=180.0/kInnoAngleUnitPerPiRad
  The minimum unit in angle.
- **kInnoAngleUnitPerDegree**=kInnoAngleUnitPerPiRad/180.0
  The number of smallest units in degrees that can be converted to the smallest units in radians.
- **kInnoChannelNumberBit**=2
  The size of kInnoChannelNumber.
- **kInnoChannelNumber**=1<<kInnoChannelNumberBit
  The maximum number of channels.
- **kInnoMaxMultiReturn**=2
  The maximum number of points received by a single laser emission = kInnoChannelNumber×InnoMaxMultiReturn.
- **kInnoVAngleDiffBase**=196
  The internal parameter.
- **kInnoNopROI**=10000.0
  The empty command for filling. The original ROI angle will be kept.

3.4 Enum

3.4.1 enum InnoLidarProtocol

The data type of the communication protocol used by LiDAR.

Syntax

```
enum InnoLidarProtocol {
  INNO_LIDAR_PROTOCOL_NONE = 0,
  INNO_LIDAR_PROTOCOL_RAW_TCP = 1,
  INNO_LIDAR_PROTOCOL_RAW_MEM = 2,
  INNO_LIDAR_PROTOCOL_PCS_TCP = 3,
  INNO_LIDAR_PROTOCOL_PCS_UDP = 4,
  INNO_LIDAR_PROTOCOL_RAW_FILE = 5,
  INNO_LIDAR_PROTOCOL_PCS_FILE = 6,
  INNO_LIDAR_PROTOCOL_MAX = 7,
};
```

Parameters

- **INNO_LIDAR_PROTOCOL_NONE**
  Initialization value.
- **INNO_LIDAR_PROTOCOL_RAW_TCP**
  Transfer raw data in TCP.
- **INNO_LIDAR_PROTOCOL_RAW_MEM**
  Transfer raw data in memory mapping.
- **INNO_LIDAR_PROTOCOL_PCS_TCP**
  Transfer data in TCP between client and pcs.
- **INNO_LIDAR_PROTOCOL_PCS_UDP**
  Transfer data in UDP between client and pcs.
- **INNO_LIDAR_PROTOCOL_RAW_FILE**
  Read raw data from the file.
- **INNO_LIDAR_PROTOCOL_PCS_FILE**

Copyright©Innovusion

Read inno raw from the file.

- **INNO_LIDAR_PROTOCOL_MAX**

  The maximum limitation of value. The value is used to confirm the validity.

3.4.2 enum InnoLidarState

The current reading status of the LiDAR stream.

Syntax

```
enum InnoLidarState {
  INNO_LIDAR_STATE_ERROR = 0,
  INNO_LIDAR_STATE_STARTING = 1,
  INNO_LIDAR_STATE_READY = 2,
  INNO_LIDAR_STATE_STREAMING = 3,
  INNO_LIDAR_STATE_UNKNOWN = 4,
  INNO_LIDAR_STATE_INVALID = 5,
};
```

Parameters

- **INNO_LIDAR_STATE_ERROR**

  Error state of the LiDAR.

- **INNO_LIDAR_STATE_STARTING**

  Stop state of the LiDAR streaming.

- **INNO_LIDAR_STATE_READY**

  Normal state of the LiDAR. Data can be obtained from the LiDAR in this state.

- **INNO_LIDAR_STATE_STREAMING**

  Default state. The state will not be called.

- **LIDAR_STATE_UNKNOWN**

  Abnormal state.

- **INNO_LIDAR_STATE_INVALID**

  Invalid state.

3.4.3 enum InnoRecorderCallbackType

Data recording callback function for distinguishing between the different data types.

Syntax

```
enum InnoRecorderCallbackType {
  INNO_RECORDER_CALLBACK_TYPE_NONE = 0,
  INNO_RECORDER_CALLBACK_TYPE_RAW = 1,
  INNO_RECORDER_CALLBACK_TYPE_RAW2 = 2,
  INNO_RECORDER_CALLBACK_TYPE_RAW3 = 3,
  INNO_RECORDER_CALLBACK_TYPE_RAW4 = 4,
  INNO_RECORDER_CALLBACK_TYPE_CALI = 5,
  INNO_RECORDER_CALLBACK_TYPE_MAX = 6,
};
```

Parameters

- **INNO_RECORDER_CALLBACK_TYPE_NONE**

  Initialization value.

- **INNO_RECORDER_CALLBACK_TYPE_RAW**

  raw data.

- **INNO_RECORDER_CALLBACK_TYPE_RAW2**

  For internal use.

- **INNO_RECORDER_CALLBACK_TYPE_RAW3**

For internal use.

- **INNO_RECORDER_CALLBACK_TYPE_RAW4**

  For internal use.

- **INNO_RECORDER_CALLBACK_TYPE_CALI**

  For internal use.

- **INNO_RECORDER_CALLBACK_TYPE_MAX**

  The maximum limitation of value. The value is used to confirm the validity.

3.4.4 enum InnoLidarMode

The LiDAR working mode.

Syntax

```
enum InnoLidarMode {
  INNO_LIDAR_MODE_NONE = 0,
  INNO_LIDAR_MODE_SLEEP = 1,
  INNO_LIDAR_MODE_STANDBY = 2,
  INNO_LIDAR_MODE_WORK_NORMAL = 3,
  INNO_LIDAR_MODE_WORK_SHORT_RANGE = 4,
  INNO_LIDAR_MODE_WORK_CALIBRATION = 5,
  INNO_LIDAR_MODE_PROTECTION = 6,
  INNO_LIDAR_MODE_WORK_QUIET = 7,
  INNO_LIDAR_MODE_WORK_INTERNAL_1 = 8,
  INNO_LIDAR_MODE_WORK_MAX = 9,
};
```

Parameters

- **INNO_LIDAR_MODE_NONE**

  Initialization value.

- **INNO_LIDAR_MODE_SLEEP**

  Sleep mode. The mode can only be used when the LiDAR is configured to wake from CAN messages.

- **INNO_LIDAR_MODE_STANDBY**

  Standby mode. In this mode, components such as the laser, polygon and Galvo are disabled. The chip keeps working for fast switching to normal mode.

- **INNO_LIDAR_MODE_WORK_NORMAL**

  Normal mode.

- **INNO_LIDAR_MODE_WORK_SHORT_RANGE**

  No longer in use.

- **INNO_LIDAR_MODE_WORK_CALIBRATION**

  Calibration mode.

- **INNO_LIDAR_MODE_PROTECTION**

  The LiDAR will be in protection mode when the laser temperature exceeds the threshold. The LiDAR will return to normal mode after the laser temperature decreases.

- **INNO_LIDAR_MODE_WORK_QUIET**

  Quiet mode. In this mode, the noise will be reduced.

- **INNO_LIDAR_MODE_WORK_INTERNAL_1**

  For Internal use. Not enabled.

- **INNO_LIDAR_MODE_WORK_MAX**

  The maximum limitation of value. The value is used to confirm the validity.

Copyright©Innovusion

3.4.5 enum InnoLidarStatus

The status of the LiDAR event.

Syntax

```
enum InnoLidarStatus {
  INNO_LIDAR_STATUS_NONE = 0,
  INNO_LIDAR_STATUS_TRANSITION = 1,
  INNO_LIDAR_STATUS_NORMAL = 2,
  INNO_LIDAR_STATUS_FAILED = 3,
  INNO_LIDAR_STATUS_MAX = 4,
};
```

Parameters

- **INNO_LIDAR_STATUS_NONE**

  Initialization value.
- **INNO_LIDAR_STATUS_TRANSITION**

  The event is in progress.
- **INNO_LIDAR_STATUS_NORMAL**

  The event execution succeeded.
- **INNO_LIDAR_STATUS_FAILED**

  The event execution failed.
- **INNO_LIDAR_STATUS_MAX**

  The maximum limitation of value. The value is used to confirm the validity.

3.4.6 enum InnoTimeSyncConfig

The time synchronization mode of the LiDAR.

Syntax

```
enum InnoTimeSyncConfig {
  INNO_TIME_SYNC_CONFIG_HOST = 0,
  INNO_TIME_SYNC_CONFIG_PTP = 1,
  INNO_TIME_SYNC_CONFIG_GPS = 2,
  INNO_TIME_SYNC_CONFIG_FILE = 3,
  INNO_TIME_SYNC_CONFIG_NTP = 4,
  INNO_TIME_SYNC_CONFIG_MAX = 5,
};
```

Parameters

- **INNO_TIME_SYNC_CONFIG_HOST**

  Time synchronization in using the LiDAR local time.
- **INNO_TIME_SYNC_CONFIG_PTP**

  PTP time synchronization.
- **INNO_TIME_SYNC_CONFIG_GPS**

  GPS time synchronization.
- **INNO_TIME_SYNC_CONFIG_FILE**

  Time synchronization using the files.
- **INNO_TIME_SYNC_CONFIG_NTP**

  NTP Time synchronization.
- **INNO_TIME_SYNC_CONFIG_MAX**

  The maximum limitation of value. The value is used to confirm the validity.

Copyright©Innovusion

3.4.7 enum InnoTimeSyncType

Time synchronization type.

Syntax

```
enum InnoTimeSyncType {
  INNO_TIME_SYNC_TYPE_NONE = 0,
  INNO_TIME_SYNC_TYPE_RECORDED = 1,
  INNO_TIME_SYNC_TYPE_HOST = 2,
  INNO_TIME_SYNC_TYPE_GPS_INIT = 3,
  INNO_TIME_SYNC_TYPE_GPS_LOCKED = 4,
  INNO_TIME_SYNC_TYPE_GPS_UNLOCKED = 5,
  INNO_TIME_SYNC_TYPE_PTP_INIT = 6,
  INNO_TIME_SYNC_TYPE_PTP_LOCKED = 7,
  INNO_TIME_SYNC_TYPE_PTP_UNLOCKED = 8,
  INNO_TIME_SYNC_TYPE_FILE_INIT = 9,
  INNO_TIME_SYNC_TYPE_NTP_INIT = 10,
  INNO_TIME_SYNC_TYPE_NTP_LOCKED = 11,
  INNO_TIME_SYNC_TYPE_NTP_UNLOCKED = 12,
  INNO_TIME_SYNC_TYPE_GPS_LOST = 13,
  INNO_TIME_SYNC_TYPE_PTP_LOST = 14,
  INNO_TIME_SYNC_TYPE_NTP_LOST = 15,
  INNO_TIME_SYNC_TYPE_MAX = 16,
};
```

Parameters

- **INNO_TIME_SYNC_TYPE_NONE**
  Initialization value.
- **INNO_TIME_SYNC_TYPE_RECORDED**
  Not currently used.
- **INNO_TIME_SYNC_TYPE_HOST**
  Synchronized time source is the host PC.
- **INNO_TIME_SYNC_TYPE_GPS_INIT**
  Switch other time synchronization types to the beginning phase of the GPS.
- **INNO_TIME_SYNC_TYPE_GPS_LOCKED**
  The GPS time synchronization is at a high confidence level.
- **INNO_TIME_SYNC_TYPE_GPS_UNLOCKED**
  The GPS time synchronization is at a low confidence level.
- **INNO_TIME_SYNC_TYPE_PTP_INIT**
  Switch other time synchronization types to the beginning phase of the PTP.
- **INNO_TIME_SYNC_TYPE_PTP_LOCKED**
  The PTP time synchronization is at a high confidence level.
- **INNO_TIME_SYNC_TYPE_PTP_UNLOCKED**
  The PTP time synchronization is at a low confidence level.
- **INNO_TIME_SYNC_TYPE_FILE_INIT**
  Playing file mode. This mode is not applicable to the scene of the LiDAR connection.
- **INNO_TIME_SYNC_TYPE_NTP_INIT**
  Switch other time synchronization types to the beginning phase of the NTP.
- **INNO_TIME_SYNC_TYPE_NTP_LOCKED**
  The NTP time synchronization is at a high confidence level.
- **INNO_TIME_SYNC_TYPE_NTP_UNLOCKED**
  The NTP time synchronization is at a low confidence level.
- **INNO_TIME_SYNC_TYPE_GPS_LOST**

Copyright©Innovusion

GPS time synchronization lost.

- **INNO_TIME_SYNC_TYPE_PTP_LOST**

  PTP time synchronization lost.
- **INNO_TIME_SYNC_TYPE_NTP_LOST**

  NTP time synchronization lost.
- **INNO_TIME_SYNC_TYPE_MAX**

  The maximum limitation of value. The value is used to confirm the validity.

3.4.8 enum InnoMessageLevel

The message level. Each message level has the same meaning as the corresponding log level.

Syntax

```
enum InnoMessageLevel
  INNO_MESSAGE_LEVEL_FATAL = 0,
  INNO_MESSAGE_LEVEL_CRITICAL = 1,
  INNO_MESSAGE_LEVEL_ERROR = 2,
  INNO_MESSAGE_LEVEL_TEMP = 3,
  INNO_MESSAGE_LEVEL_WARNING = 4,
  INNO_MESSAGE_LEVEL_DEBUG = 5,
  INNO_MESSAGE_LEVEL_INFO = 6,
  INNO_MESSAGE_LEVEL_TRACE = 7,
  INNO_MESSAGE_LEVEL_DETAIL = 8,
  INNO_MESSAGE_LEVEL_MAX = 9,
};
```

Parameters

- **INNO_MESSAGE_LEVEL_FATAL**

  Fatal and unrecoverable error. The fatal error will cause the exit of the application.
- **INNO_MESSAGE_LEVEL_CRITICAL**

  Critical and irreversible error. When this error occurs, the system cannot work properly.
- **INNO_MESSAGE_LEVEL_ERROR**

  Error and abnormal message.
- **INNO_MESSAGE_LEVEL_TEMP**

  Not currently used.
- **INNO_MESSAGE_LEVEL_WARNING**

  Warning message. This message is used to alert that there may be potential errors.
- **INNO_MESSAGE_LEVEL_DEBUG**

  Debug message. The message helps debug the application.
- **INNO_MESSAGE_LEVEL_INFO**

  Message about the application operation.
- **INNO_MESSAGE_LEVEL_TRACE**

  Trace message. The message is only used for debugging and outputting details.
- **INNO_MESSAGE_LEVEL_DETAIL**

  Detail message. The message is only used for debugging and outputting more details.
- **INNO_MESSAGE_LEVEL_MAX**

  The maximum limitation of value. The value is used to confirm the validity.

3.4.9 enum InnoMessageCode

Events that generate the message data.

Syntax

```
enum InnoMessageCode {
  INNO_MESSAGE_CODE_NONE = 0,
  INNO_MESSAGE_CODE_LIB_VERSION_MISMATCH,
  INNO_MESSAGE_CODE_READ_TIMEOUT,
  INNO_MESSAGE_CODE_CANNOT_READ,
  INNO_MESSAGE_CODE_BAD_CONFIG_YAML,
  INNO_MESSAGE_CODE_OVERHEAT_PROTECTION,
  INNO_MESSAGE_CODE_TO_NON_WORKING_MODE,
  INNO_MESSAGE_CODE_READ_FILE_END,
  INNO_MESSAGE_CODE_RAW_RECORDING_FINISHED,
};
```

Parameters

- **INNO_MESSAGE_CODE_NONE**

  Initialization value.
- **INNO_MESSAGE_CODE_LIB_VERSION_MISMATCH**

  The API version does not match the SDK version.
- **INNO_MESSAGE_CODE_READ_TIMEOUT**

  The LiDAR reads data timeout.
- **INNO_MESSAGE_CODE_CANNOT_READ**

  Cannot read data from LiDAR.
- **INNO_MESSAGE_CODE_BAD_CONFIG_YAML**

  The Yaml file format is incorrect.
- **INNO_MESSAGE_CODE_OVERHEAT_PROTECTION**

  Enter protection mode because of the LiDAR overheating.
- **INNO_MESSAGE_CODE_TO_NON_WORKING_MODE**

  The LiDAR switches to non-work mode.
- **INNO_MESSAGE_CODE_READ_FILE_END**

  Read to the end of the file. It usually occurs when using a file as the data source.
- **INNO_MESSAGE_CODE_RAW_RECORDING_FINISHED**

  Recording raw data is complete.

3.4.10 enum InnoFrameDirection

The direction of Galvo motion.

Syntax

```
enum InnoFrameDirection {
  INNO_FRAME_DIRECTION_DOWN = 0,  /* top->bottom   */
  INNO_FRAME_DIRECTION_UP = 1,    /* bottom -> top */
  INNO_FRAME_DIRECTION_MAX = 2,
};
```

Parameters

- **INNO_FRAME_DIRECTION_DOWN**

  The Galvo scans from top to bottom.
- **INNO_FRAME_DIRECTION_UP**

  The Galvo scans from bottom to top.
- **INNO_FRAME_DIRECTION_MAX**

  The maximum limitation of value. The value is used to confirm the validity.

Copyright©Innovusion

3.4.11 enum InnoItemType

The data type carried by InnoDataPacket.

Syntax

```
enum InnoItemType {
  INNO_ITEM_TYPE_NONE = 0,
  INNO_ITEM_TYPE_SPHERE_POINTCLOUD = 1,
  INNO_ITEM_TYPE_MESSAGE = 2,
  INNO_ITEM_TYPE_MESSAGE_LOG = 3,
  INNO_ITEM_TYPE_XYZ_POINTCLOUD = 4,
  INNO_ITEM_TYPE_MAX = 5,
};
```

Parameters

- **INNO_ITEM_TYPE_NONE**

  Initialization value.
- **INNO_ITEM_TYPE_SPHERE_POINTCLOUD**

  Carry InnoChannelPoint.
- **INNO_ITEM_TYPE_MESSAGE**

  Carry InnoMessage.
- **INNO_ITEM_TYPE_MESSAGE_LOG**

  Carry log data in string format.
- **INNO_ITEM_TYPE_XYZ_POINTCLOUD**

  Carry 3D coordinate axis point data in InnoXyzPoint.
- **INNO_ITEM_TYPE_MAX**

  The maximum limitation of value. The value is used to confirm the validity.

3.4.12 enum InnoReflectanceMode

The LiDAR reflectance mode.

Syntax

```
enum InnoReflectanceMode {
  INNO_REFLECTANCE_MODE_NONE = 0,
  INNO_REFLECTANCE_MODE_INTENSITY = 1,
  INNO_REFLECTANCE_MODE_REFLECTIVITY = 2,
  INNO_REFLECTANCE_MODE_MAX = 3,
};
```

Parameters

- **INNO_REFLECTANCE_MODE_NONE**

  Initialization value.
- **INNO_REFLECTANCE_MODE_INTENSITY**

  The intensity mode.
- **INNO_REFLECTANCE_MODE_REFLECTIVITY**

  The reflectivity mode.
- **INNO_REFLECTANCE_MODE_MAX**

  The maximum limitation of value. The value is used to confirm the validity.

3.4.13 enum InnoConfidenceLevel

The presence of the abnormal LiDAR pointcloud would affect the confidence level of the pointcloud data. The enumeration values indicate the confidence level of the LiDAR pointcloud data.

Syntax

```
enum InnoConfidenceLevel {
  INNO_NO_CONFIDENCE = 0,
  INNO_RARE_CONFIDENCE = 1,
  INNO_WELL_CONFIDENCE = 2,
  INNO_FULL_CONFIDENCE = 3,
};
```

Parameters

- **INNO_NO_CONFIDENCE**

  The lowest confidence level.
- **INNO_RARE_CONFIDENCE**

  Low confidence level.
- **INNO_WELL_CONFIDENCE**

  High confidence level.
- **INNO_FULL_CONFIDENCE**

  Highest confidence level.

3.4.14 enum InnoMultipleReturnMode

The return mode of the LiDAR. The number and type of echoes received after a single laser emission of the LiDAR.

Syntax

```
enum InnoMultipleReturnMode {
  INNO_MULTIPLE_RETURN_MODE_NONE = 0,
  INNO_MULTIPLE_RETURN_MODE_SINGLE = 1,
  INNO_MULTIPLE_RETURN_MODE_2_STRONGEST = 2,
  // one strongest return and one furthest return
  INNO_MULTIPLE_RETURN_MODE_2_STRONGEST_FURTHEST = 3,
  INNO_MULTIPLE_RETURN_MODE_MAX
};
```

Parameters

- **INNO_MULTIPLE_RETURN_MODE_NONE**

  Initialization value.
- **INNO_MULTIPLE_RETURN_MODE_SINGLE**

  Take the point with the strongest reflection. (The single return mode: strongest)
- **INNO_MULTIPLE_RETURN_MODE_2_STRONGEST**

  Take the points with the strongest and second strongest reflection. (The dual return mode: strongest + 2 strongest)
- **INNO_MULTIPLE_RETURN_MODE_2_STRONGEST_FURTHEST**

  Take the points with the strongest and the furthest reflection. (The dual return mode: strongest + furthest)
- **INNO_MULTIPLE_RETURN_MODE_MAX**

  The maximum limitation of value. The value is used to confirm the validity.

3.4.15 enum InnoGalvoMode

The scanning mode of the Galvo.

Syntax

Copyright©Innovusion

```
enum InnoGalvoMode {
  INNO_GALVO_MODE_NONE = 0,
  INNO_GALVO_MODE_NORMAL,
  INNO_GALVO_MODE_FLYBACK,
  INNO_GALVO_MODE_MAX
};
```

Parameters

- **INNO_GALVO_MODE_NONE**

  Initialization value.
- **INNO_GALVO_MODE_NORMAL**

  The normal mode of the Galvo. The Galvo scans back and forth.
- **INNO_GALVO_MODE_FLYBACK**

  Flyback mode of the Galvo. After scanning once, the Galvo returns to the original position to continue scanning.
- **INNO_GALVO_MODE_MAX**

  The maximum limitation of value. The value is used to confirm the validity.

3.4.16 enum InnoLogLevel

SDK log level. The head file path: src/utils/inno_lidar_log.h.

Syntax

```
enum InnoLogLevel {
  INNO_LOG_LEVEL_FATAL = 0,
  INNO_LOG_LEVEL_CRITICAL = 1,
  INNO_LOG_LEVEL_ERROR = 2,
  INNO_LOG_LEVEL_TEMP = 3,
  INNO_LOG_LEVEL_WARNING = 4,
  INNO_LOG_LEVEL_DEBUG = 5,
  INNO_LOG_LEVEL_INFO = 6,
  INNO_LOG_LEVEL_TRACE = 7,
  INNO_LOG_LEVEL_DETAIL = 8,
  INNO_LOG_LEVEL_MAX = 9,
};
```

Parameters

- **INNO_LOG_LEVEL_FATAL**

  Fatal and unrecoverable error. The fatal error will cause the exit of the application.
- **INNO_LOG_LEVEL_CRITICAL**

  Critical and irreversible error. When this error occurs, the system cannot work properly.
- **INNO_LOG_LEVEL_ERROR**

  Error message.
- **INNO_LOG_LEVEL_TEMP**

  Not currently used.
- **INNO_LOG_LEVEL_WARNING**

  Warning message. This message is used to alert that there may be potential errors.
- **INNO_LOG_LEVEL_DEBUG**

  Debug message. The message helps debug the application.
- **INNO_LOG_LEVEL_INFO**

  Message about the application running.
- **INNO_LOG_LEVEL_TRACE**

  Trace message. The message is only used for debugging and outputting details.

- **INNO_LOG_LEVEL_DETAIL**

  Detail message. The message is only used for debugging and outputting more details.
- **INNO_LOG_LEVEL_MAX**

  The maximum limitation of value. The value is used to confirm the validity.

3.5 Structure

3.5.1 InnoBlockHeader

BlockHeader data structure. The size is 17 bytes.

Syntax

```
Struct  __attribute__((packed)) InnoBlockHeader {
  /* horizontal angle, 0 is straight forward, right is positive,
     unit is kRadPerInnoAngleUnit rad, range (-PI to PI) */
  int16_t h_angle;

  /* vertical angle, 0 is the horizon, up is positive,
     unit is kRadPerInnoAngleUnit rad, range (-PI to PI) */
  int16_t v_angle;

/* relative timestamp (to ts_start_us) in 10us, 0-655,350us */
  uint16_t ts_10us;
  uint16_t scan_idx;     /* point idx within the scan line */
  uint16_t scan_id: 9;   /* id of the scan line */

// real angle is h_angle + h_angle_diff_1
  int64_t h_angle_diff_1: 9;
  int64_t h_angle_diff_2: 10;
  int64_t h_angle_diff_3: 11;

// real angle is v_angle + v_angle_diff_1 + kVAngleDiffBase * channel
  int64_t v_angle_diff_1: 8;  // 196 + [-128, 127]
  int64_t v_angle_diff_2: 9;  // 392 + [-256, 255]
  int64_t v_angle_diff_3: 9;  // 588 + [-256, 255]

  /*   0: in sparse region
    0x01: in vertical slow region
    0x10: in horizontal slow region
    0x11: in center ROI */
  uint64_t in_roi: 2;
  uint64_t facet: 3;
  uint64_t reserved_flags: 2; /* all 0 */
};
```

Members

- **h_angle**

  Horizontal FoV. h_angle =0 means the FoV is straight ahead. Right is the positive direction. The unit is rad. The range is -PI to PI.
- **v_angle**

  Vertical FoV. v_angle =0 means the FoV is up forward. Upward is the positive direction. The unit is rad. The range is -PI to PI.
- **ts_10us**

  Relative timestamp. (to ts_start_us). The unit is 10 microseconds. The range is 0 to 655350 ms.
- **scan_idx**

  The index of a point on the scanning line.
- **scan_id**

  The index of a scanning line.

Copyright©Innovusion

- **h_angle_diff_1**

  The actual angle = horizontal angle + horizontal angle deviation 1.
- **h_angle_diff_2**

  The actual angle = horizontal angle + horizontal angle deviation 2.
- **h_angle_diff_3**

  The actual angle = horizontal angle + horizontal angle deviation 3.
- **v_angle_diff_1**

  The actual angle = vertical angle + vertical angle deviation 1 + kVAngleDiffBase× channel 196+ [-128,127].
- **v_angle_diff_2**

  The actual angle = vertical angle + vertical angle deviation 2 + kVAngleDiffBase× channel 392+ [-256,255].
- **v_angle_diff_3**

  The actual angle = vertical angle + vertical angle deviation 2 + kVAngleDiffBase× channel 588+ [-256,255].
- **in_roi**

  The position of the point is relative to the ROI.

  in_roi=0x00: in sparse region

  in_roi=0x11: in center of ROI
- **facet**

  The serial number of the mirrors in the Block. (There are 5 mirrors, and the serial number is 0 to 4)
- **reserved_flags**

  All 0.

3.5.2 InnoChannelPoint

ChannelPoint data structure. The size is 4 bytes.

Syntax

```
Struct InnoChannelPoint {
  uint32_t radius: 17;     /* distance in distance unit, range [0, 655.35m] */
  uint32_t refl: 9;        /* reflectance, 1-254, 255 means a reflector     */
                            /* or intensity, also 1-254 & 255=reflector      */
  uint32_t is_2nd_return: 1;        /* 0: 1st return, 1:2nd return            */
  uint32_t type: 2;        /* 0: normal, 1: ground, 2: fog                  */
  uint32_t elongation: 4;  /* elongation */
};
```

Members

- **radius**

  The distance of the point. The range is 0 to 655.35m. The unit is 1/200m.
- **refl**

  Reflectivity or intensity. The range is 1 to 254. The refl=255 means the total reflectance.
- **is_2nd_return**

  is_2nd_retun=1 means the dual return mode. is_2nd_retun=0 means the single return mode.
- **type**

  The reflectance type.
- **elongation**

  The pulse width corresponds to the echoes.

3.5.3 InnoBlock

The points in the same block are saved in one InnoBlock, such as InnoBlock1. The points of 4 channels received by the same laser emission are saved in the single return mode.

Syntax

```
Struct InnoBlock {
  InnoBlockHeader header;
  InnoChannelPoint points[0];
};

Struct InnoBlock1 {
  InnoBlockHeader header;
  InnoChannelPoint points[kInnoChannelNumber];
};

Struct InnoBlock2 {
  InnoBlockHeader header;
  InnoChannelPoint points[kInnoChannelNumber * kInnoMaxMultiReturn];
};
```

Members

- **header**

  Block head.

- **points**

  Pointcloud data.

Remarks

InnoBlock1: Single echo.

InnoBlock2： Multiple echoes.

3.5.4 InnoCommonHeader

CommonHeader data structure. The size is 26 bytes.

Syntax

```
Struct InnoCommonHeader {
  /* 6 bytes */
  InnoCommonVersion version;

  /* 4 bytes, cover every thing except checksum */
  uint32_t checksum;

  /* 4 bytes */
  uint32_t size;

  /* 2 bytes */
  uint16_t source_id :4;           /* up to 16 different LiDAR source */
  uint16_t timestamp_sync_type :4; /* enum InnoTimestampSyncType      */
  uint16_t reserved :8;

  /* 8 bytes */
  InnoTimestampUs ts_start_us; /* epoch time of start of frame, in micro-sec */

  /* 2 bytes */
  uint8_t lidar_mode;        /* enum InnoLidarMode    */
  uint8_t lidar_status;      /* enum InnoLidarStatus  */
};
```

Members

- **version**

  The version information of the data packet. For details, refer to 3.5.9 InnoCommonVersion.

Copyright©Innovusion

- **checksum**

  Checksum. Used for CRC checking.
- **size**

  The size of InnoDataPacket.
- **source_id**

  Identify different LiDARs when there are multiple LiDARs. A default value of 0 when there is only one LiDAR.
- **timestamp_sync_type**

  The time synchronization type. For the detailed values of time synchronization type, refer to 3.4.7 enum InnoTimeSyncType.
- **reserved**

  Reserved filed. Currently not used.
- **ts_start_us**

  The trigger time of the laser pulse in the 0th Block of InnoDataPacket.
- **lidar_mode**

  The working mode of the LiDAR. For the detailed values of working mode, refer to 3.4.4 enum InnoLidarMode.
- **lidar_status**

  The status of the LiDAR. For the detailed values of LiDAR status, refer to 3.4.5 enum InnoLidarStatus.

3.5.5 InnoDataPacket

DataPacket data structure. The size is 54+X bytes.

Syntax

```
Struct InnoDataPacket {
  InnoCommonHeader common;

  /* 12 bytes */
  uint64_t idx;          /* frame index, start from 0                     */
  uint16_t sub_idx;      /* sub-frame index, start from 0 for every frame */
  uint16_t sub_seq;      /* sequence within a sub-frame                   */

  /* 10 byte */
  /* type in enum InnoItemType, each type uses independent global idx */
  uint32_t type :8;
  uint32_t item_number :24;        /* max 4 * 1024 * 1024               */
  uint16_t item_size;              /* max 65535, 0 means variable size  */
  uint32_t topic;                  /* reserved                          */

  /* 2 bytes */
  uint16_t scanner_direction :1; /* 0: top->bottom, 1: bottom->top          */
  uint16_t use_reflectance   :1; /* 0: intensity mode, 1: reflectance mode  */
  uint16_t multi_return_mode :3; /* ... */
  uint16_t confidence_level  :2; /* 0: no confidence, 3: higest             */
  uint16_t is_last_sub_frame :1; /* 1: the last sub frame of a frame        */
  uint16_t is_last_sequence  :1; /* 1: the last piece of a sub frame        */
  uint16_t has_tail :1;          /* has additional tail struct after points */
  uint16_t reserved_flag :6;     /* all 0 */

  /* 4 bytes */
  int16_t roi_h_angle;           /* configured ROI in InnoAngleUnit */
  int16_t roi_v_angle;

  union {
    char c[0];
```

```
      InnoBlock1 inno_block1s[0];
      InnoBlock2 inno_block2s[0];
      InnoMessage messages[0];
      InnoXyzPoint xyz_points[0];
    };
  };
```

Members

- **common**

  Refer to 3.5.4 InnoCommonHeader.

- **idx**

  The serial number of the frame. The serial number starts from 0.

- **sub_idx**

  There are multiple InnoDataPacket packets in each frame. sub_idx is the serial number of InnoDataPacket packet in a frame.

- **sub_seq**

  The serial number of the sub frame. Currently not used. The value is 0 by default.

- **type**

  The data type of the InnoDataPacket. For details, refer to 3.4.11 enum InnoItemType.

- **item_number**

  The number of Blocks in the current InnoDataPacket. The maximum is 4×1024×1024.

- **item_size**

  The size of a single Block. The maximum is 65535. The value of item_size is 0 means the size is flexible.

- **topic**

  Currently not used. The value is 0 by default.

- **scanner_direction**

  The direction of the laser scanning. For the detailed values of scanning direction, refer to 3.4.10 enum InnoFrameDirection.

- **use_reflectance**

  The reflectance mode. For the detailed values of reflectance mode, refer to 3.4.12 enum InnoReflectanceMode.

- **multi_return_mode**

  The return mode. For the detailed values of return mode, refer to 3.4.14 enum InnoMultipleReturnMode.

- **confidence_level**

  The confidence level of the pointcloud data. For the detailed values of return mode, refer to 3.4.13 enum InnoConfidenceLevel.

- **is_last_sub_frame**

  Determine if it is the last InnoDataPacket in a frame.

  is_last_sub_frame=0: Not the last InnoDataPacket.

  is_last_sub_frame=1: The last InnoDataPacket.

- **is_last_sequence**

  is_last_sequence=1: The last piece of in a subframe.

- **has_tail**

  Currently not used. The value is 1 by default.

- **reserved_flag**

Copyright©Innovusion

Reserved. The value is 0. Currently not used.

- **roi_h_angle**

  The horizontal angle of the ROI center.

- **roi_v_angle**

  The vertical angle of the ROI center.

- **union {...}**

  Pointcloud data. The data type is determined according to its member. For details, refer to 3.4.11 enum InnoItemType.

3.5.6 InnoStatusCounters

StatusCounters data structure. The size is 320 bytes.

Syntax

```
Struct InnoStatusCounters {
  uint64_t point_data_packet_sent;
  uint64_t point_sent;
  uint64_t message_packet_sent;
  uint64_t raw_data_read;
  uint64_t total_frame;
  uint64_t total_polygon_rotation;
  uint64_t total_polygon_facet;
  uint32_t power_up_time_in_second;
  uint32_t process_up_time_in_second;
  uint32_t lose_ptp_sync;
  uint32_t bad_data[4];
  uint32_t data_drop[8];
  uint32_t in_signals[8];
  uint16_t latency_10us_average[6];
  uint16_t latency_10us_variation[6];
  uint16_t latency_10us_max[6];
  uint32_t big_latency_frame;
  uint32_t bad_frame;
  uint32_t big_gap_frame;
  uint32_t small_gap_frame;
  uint16_t cpu_percentage;
  uint16_t mem_percentage;
  uint16_t motor[5];  /* std,min,max1,max2 */
  uint16_t galvo[5];  /* std,min,max1,max2 */
  uint16_t netstat_rx_speed_kBps;
  uint16_t netstat_tx_speed_kBps;
  uint16_t netstat_rx_drop;
  uint16_t netstat_tx_drop;
  uint16_t netstat_rx_err;
  uint16_t netstat_tx_err;
  uint16_t sys_cpu_percentage[4];
  uint32_t reserved[19];
};
```

Members

- **point_data_packet_sent**

  The number of InnoDataPackets sent.

- **point_sent**

  The number of InnoBlocks sent.

- **message_packet_sent**

  The number of messages sent.

- **raw_data_read**

  The size of raw data read.

- **total_frame**
  The number of frames sent.
- **total_polygon_rotation**
  The number of rotations done by the polygon as calculated by encoder.
- **total_polygon_facet**
  The number of rotations done by the polygon as calculated by stage angle and new line detection.
- **power_up_time_in_second**
  Power-up time. The unit is second.
- **process_up_time_in_second**
  Execution time. The unit is second.
- **lose_ptp_sync**
  The number of times “PTP time synchronization” was lost.
- **bad_data[4]**
  Bad data count. Currently not used. The value is 0 by default.
- **data_drop[8]**
  Data loss count.
- **signals[8]**
  Signal count.
- **latency_10us_average[6]**
  Average delay. The unit is 10us.
- **latency_10us_variation[6]**
  Fluctuation delay. The unit is 10us.
- **latency_10us_max[6]**
  Maximum delay. The unit is 10us.
- **big_latency_frame**
  Long delay frame count. Currently not used. The value is 0 by default.
- **bad_frame**
  Bad frame count. Currently not used. The value is 0 by default.
- **big_gap_frame**
  Long gap frame count. Currently not used. The value is 0 by default.
- **small_gap_frame**
  Short gap frame count. Currently not used. The value is 0 by default.
- **cpu_percentage**
  CPU utilization.
- **mem_percentage**
  Memory utilization.
- **motor[5]**
  Statistics related to the motor speed. The five values are the standard deviation, the minimum value, the maximum value, the maximum difference between two adjacent values, and the average value of the motor speed.
- **galvo[5]**

Copyright©Innovusion

Statistics related to the Galvo speed. The five values are the standard deviation, the minimum value, the maximum value, the maximum difference between two adjacent values, and the average value of the Galvo speed.

- **netstat_rx_speed_kBps**

  The size of the data received per second.
- **netstat_tx_speed_kBps**

  The size of the data sent per second.
- **netstat_rx_drop**

  The number of lost packets in the network reception.
- **netstat_tx_drop**

  The number of lost packets in the network delivery.
- **netstat_rx_err**

  The number of error packets in the network reception.
- **netstat_tx_err**

  The number of error packets in the network delivery.
- **sys_cpu_percentage[4]**

  The total utilization in system CPU.
- **reserved[24]**

  Reserved. The default value is 0.

Remarks

Netstat is the program that accesses the network connection status and the related information in the kernel.

3.5.7 InnoStatusPacket

StatusPacket data structure. The size is 714 bytes.

Syntax

```
Struct InnoStatusPacket {
  static const uint32_t kSnSize = 16;
  InnoCommonHeader common;

  uint64_t idx;  /* global index of all InnoStatusPacket */

  uint8_t status_packet_interval_ms;  /* status packet send interval in ms    */
  uint8_t pre_lidar_mode;             /* previous InnoLidarMode               */
  uint16_t in_transition_mode_ms;  /* time (ms), LiDAR in the transition mode */

  char sn[kSnSize];                /* lidar serial number */
  uint16_t fault_version;
  uint16_t reserve;
  uint32_t reserved[3];

  InnoStatusInFaults in_faults;
  InnoStatusExFaults ex_faults;
  InnoStatusCounters counters;
  InnoStatusSensorReadings sensor_readings;
};
```

Members

- **Common**

  Refer to 3.5.4 InnoCommonHeader.
- **idx**

  The serial number of the status packet. The serial number starts from 0.
- **status_packet_interval_ms**

  The time interval between the status packets sent. The unit is microseconds.

- **pre_lidar_mode**
  The previous mode of the LiDAR. Refer to 3.4.15 enum InnoLidarMode.
- **in_transition_mode_ms**
  Time required for the LiDAR mode switching. The unit is microseconds.
- **sn[kSnSize]**
  The serial number of the LiDAR.
- **fault_version**
  The number of times the fault status changed. The initial value is 1.
- **reserved[4]**
  Reserved.
- **in_faults**
  Internal fault code. Refer to 3.5.8 InnoStatusInFaults.
- **ex_faults**
  Currently not used. The value is 0 by default.
- **counters**
  Data Statistics. Refer to 3.5.6 InnoStatusCounters.
- **sensor_readings**
  Sensor information. Refer to 3.5.10 InnoStatusSensorReadings.

3.5.8 InnoStatusInFaults

The data structure related to Faults. Used to store the Fault status. The size is 8 bytes.

Syntax

```
struct InnoStatusInFaults {
  bool no_fault(void) const {
    return faults == 0;
  }
  void clear_faults(void) {
    faults = 0;
  }
  bool no_fault(enum InnoLidarInFault c) const {
    return !((faults >> c) & 1);
  }
  bool no_fault_except(enum InnoLidarInFault c) const {
    return faults == 0 || ((faults & ~(1LU << c)) == 0);
  }
  void set_fault(enum InnoLidarInFault c) {
    faults |= (1LU << c);
  }
  void clear_fault(enum InnoLidarInFault c) {
    faults &= ~(1LU << c);
  }
  /*
   *  Each bit represent one fault defined in enum InnoLidarInFault.
   *  0 means no fault.
   */
  uint64_t faults;
};
```

Members

- **faults**
  Each bit represents a Fault value. The specific meaning of the Fault is as follows.

Table 4 LiDAR failure Description

Fault	Description
INNO_LIDAR_IN_FAULT_POWER_LOW=1	The battery undervoltage.
INNO_LIDAR_IN_FAULT_POWER_HIGH=2	The battery overvoltage.

Copyright©Innovusion

INNO_LIDAR_IN_FAULT_WINDOW_BLOCKAGE1=3	The LiDAR window is obscured.
INNO_LIDAR_IN_FAULT_LASER_INTERLOCK=7	The laser is powering up. The cover is open while working. The polygon RPM is below the threshold.
INNO_LIDAR_IN_FAULT_COMM_LASER=8	UART communication failure between the laser and SoC: Checksum/Timeout.
INNO_LIDAR_IN_FAULT_LASER=9	Internal laser fault, such as overheating, pump overcurrent, trigger fault, and power fault.
INNO_LIDAR_IN_FAULT_COMM_DSP=10	UART/SPI communication failure between the DSP and SoC: Checksum/Timeout.
INNO_LIDAR_IN_FAULT_DSP =12	DSP chip fault, such as clock fault, memory fault, CPU fault, ADC fault, etc.
INNO_LIDAR_IN_FAULT_POLYGON_CONTROL=13	● Motor control fault, such as varying rotation speed, motor blockage, U/V/W-phase current overcurrent and DC overvoltage. ● Polygon driver fault, such as overcurrent, overheating, and performance failures.
INNO_LIDAR_IN_FAULT_POLYGON_SENSOR=14	● Polygon encoder fault, such as short circuit to the ground/ short circuit to the power/ open circuit of the signal lines, out of range/stuck/oscillated of the speed signal. ● Faults related to the polygon hall.
INNO_LIDAR_IN_FAULT_GALVO_CONTROL=15	● Galvo motor control fault, such as varying rotation speed and Galvo blockage. ● Galvo driver fault, such as overcurrent, overheating, and performance failure.
INNO_LIDAR_IN_FAULT_GALVO_SENSOR=16	Galvo encoder fault, such as short circuit to the ground/ short circuit to the power supply/ open circuit of the signal cable, inconsistency of dual output current signals.
INNO_LIDAR_IN_FAULT_OPTIC1=17	Refers to the open circuit of the optical path. (Such as the optical fiber disconnection of the reference optical path, detection board fault, etc.)
INNO_LIDAR_IN_FAULT_OPTIC2=18	The open circuit of the main optical path. (open circuit of the main optical path, detection board fault, etc.)
INNO_LIDAR_IN_FAULT_IIC_DSP=19	IIC communication failure between the DSP and peripherals (except SoC).
INNO_LIDAR_IN_FAULT_IIC_SoC=20	IIC communication failure between the SoC and peripherals (except DSP).
INNO_LIDAR_IN_FAULT_COMM_ADC=25	Communication failure between the ADC chip and SoC: SPI and JESD204C.
INNO_LIDAR_IN_FAULT_SoC=27	SoC chip fault, such as clock fault, memory fault, CPU fault, etc.
INNO_LIDAR_IN_FAULT_SOC_EXTWD=28	SoC external watchdog fault

INNO_LIDAR_IN_FAULT_RAWDATA_STREAM=29	Raw data order error, such as there is no Galvo encoder between two polygon encoder signals, etc.
INNO_LIDAR_IN_FAULT_POLYGON_TO=30	Polygon encoder signal timeout.
INNO_LIDAR_IN_FAULT_GALVO_TO=31	Galvo encoder signal timeout.
INNO_LIDAR_IN_FAULT_TRIGGER_TO=32	Laser trigger signal timeout.
INNO_LIDAR_IN_FAULT_POWSUPL1=33	DSP Safe PMIC chip fault.
INNO_LIDAR_IN_FAULT_POWSUPL2=34	SoC PMIC chip fault.
INNO_LIDAR_IN_FAULT_LPDDR4=35	LPDDR4 chip fault, such as reading/writing data errors, data damage.
INNO_LIDAR_IN_FAULT_FLASH=36	FLASH fault, such as reading/writing data errors, data damage, and SPI communication fault.
INNO_LIDAR_IN_FAULT_NETWORK1=37	Ethernet receiving message fault, PTP timeout, TCP request CRC fault
INNO_LIDAR_IN_FAULT_NETWORK2=38	EPHY chip fault.
INNO_LIDAR_IN_FAULT_OVERHEAT1=39	The laser temperature rose to the 1-level threshold of 95℃ (203°F).
INNO_LIDAR_IN_FAULT_OVERHEAT2=40	The laser temperature rose to the 2-level threshold of 99℃ (210.2°F).
INNO_LIDAR_IN_FAULT_OVERHEAT3=41	The laser temperature rose to the 3-level threshold of 100℃ (212°F).
INNO_LIDAR_IN_FAULT_CONFIG1=42	Software configuration data error, such as calibration data fault, upper layer application data fault and UDS configuration data fault.
INNO_LIDAR_IN_FAULT_CONFIG2=43	Firmware configuration data error.
INNO_LIDAR_IN_FAULT_ASSERT_FAILURE=44	Software assertion error.
INNO_LIDAR_IN_FAULT_CPULOAD_HIGH=45	Excessive CPU load.
INNO_LIDAR_IN_FAULT_LATENCY_LONG=46	Excessive system operation delay.
INNO_LIDAR_IN_FAULT_RESERVED07=47	Active data throwing in the Signal stage.
INNO_LIDAR_IN_FAULT_EXCESSIVE_NOISE=49	Excessive noise is found in the software.
INNO_LIDAR_IN_FAULT_DATA_DROP1 50	Software throws data actively in the noise filtering stage.
INNO_LIDAR_IN_FAULT_DATA_DROP2=51	Software throws data actively in the pointcloud sending stage.
INNO_LIDAR_IN_FAULT_DATA_DROP3=52	Throw data actively in the SoC PL processing data stage.
INNO_LIDAR_TEMPHIGH_INHIBIT=53	LiDAR is in protection mode.

Copyright©Innovusion

INNO_LIDAR_IN_FAULT_REFINTENSITY=55	Refers to abnormal light intensity.

3.5.9 InnoCommonVersion

CommonVersion data structure. Used to indicate the firmware and software version. The size is 6 bytes.

Syntax

```
struct InnoCommonVersion {
  /* 2 byte */
  uint16_t magic_number;
  /* 2 byte */
  uint8_t major_version;
  uint8_t minor_version;
  /* 2 byte, all 0 */
  uint16_t reserved;
};
```

Members

- **magic_number**
  The magic number. Used to verify the data.
- **major_version**
  The major version.
- **minor_version**
  The minor version.
- **reserved**
  Reserved.

3.5.10 InnoStatusSensorReadings

StatusSensorReadings data structure. The size is 20 bytes.

Syntax

```
/* 208 bytes InnoStatusSensorReadings */
struct InnoStatusSensorReadings {
  int16_t temperature_fpga_10th_c;
  int16_t temperature_laser_10th_c;
  int16_t temperature_adc_10th_c;
  int16_t temperature_board_10th_c;
  int16_t temperature_det_10th_c[4];
  int16_t temperature_other_10th_c[3];
  uint16_t heater_current_ma;

  uint32_t motor_rpm_1000th;          /* polygon rpm * 1000   */
  uint32_t galvo_fpm_1000th;          /* frame per min * 1000 */
  uint64_t motor_rotation_total;
  uint64_t galvo_round_total;
  uint16_t moisture_index[2];         /* moisture index        */
  uint16_t window_blockage_index[2];  /* window blockage index */
  uint16_t motor[6];  /* ma */
  uint16_t galvo[6];  /* ma */
  uint16_t laser[6];  /* ma */
  uint16_t galvo_status_client;        /* set in client sdk */
  uint16_t galvo_offset_angle_client;  /* set in client sdk */
  uint16_t reserved[48];
};
```

Members

- **temperature_fpga_10th_c**
  FPGA temperature. The unit is 0.1 degrees Celsius. For example, if the FPGA temperature is 30°C, then temperature_fpga_10th_c=300.

- **temperature_laser_10th_c**
  Laser temperature. The unit is 0.1 degrees Celsius. For example, if the laser temperature is 30°C, then temperature_laser_10th_c=300.
- **temperature_adc_10th_c**
  ADC module temperature. The unit is 0.1 degrees Celsius. For example, if the ADC module temperature is 30°C, then temperature_adc_10th_c=300.
- **temperature_board_10th_c**
  Motherboard temperature. The unit is 0.1 degrees Celsius. For example, if the motherboard temperature is 30°C, temperature_board_10th_c=300.
- **temperature_det_10th_c**
  Detection board temperature. The unit is 0.1 degrees Celsius. The detection board is used to detect the fault. For example, if the detection board temperature is 30°C, then temperature_det_10th_c=300=300.
- **temperature_other_10th_c**
  Reserved.
- **heater_current_ma**
  Thermal current. The unit is mA.
- **motor_rpm_1000th**
  Polygon rotation speed, the unit is RPM/1000. For example, if the rotation speed is 4200 RPM, then motor_rpm_1000th=4200000.
- **galvo_fpm_1000th**
  Galvo scan speed. The unit is RPM/1000. For example, if the scan speed is 10 RPM, then galvo_fpm_1000th=10000.
- **motor_rotation_total**
  The total number of polygon rotations.
- **galvo_round_total**
  The total number of Galvo scanning iterations.
- **moisture_index**
  Reserved. The value is 0 by default.
- **window_blockage_index**
  Reserved. The value is 0 by default.
- **Motor**
  Polygon current. The unit is mA.
- **galvo**
  Galvo current. The unit is mA.
- **laser**
  Laser current. The unit is mA.
- **galvo_status_client**
  Customized parameters.
- **galvo_offset_angle_client**
  Customized parameters.
- **Reserved**
  Reserved.

3.5.11 InnoMessage

Message packet.

Syntax

Copyright©Innovusion

```
struct InnoMessage {
  uint32_t size;  // size of the whole InnoMessage,
                  //   i.e. size of content + sizeof(InnoMessage)
  uint32_t src;
  uint64_t id;
  uint32_t level;      /* enum InnoMessageLevel */
  uint32_t code;       /* message code          */
  int32_t reserved[4]; /* all 0                 */
  char content[0];     /* 0 end string          */
};
```

Members

- **size**
  The actual memory size of the InnoMessage. Size= the size of content + sizeof(InnoMessage).
- **src**
  The source of a message. The return value is the LiDAR handle.
- **id**
  Message number. The number will always be increasing.
- **level**
  Refer to 3.4.8 enum InnoMessageLevel.
- **code**
  Refer to 3.4.9 enum InnoMessageCode.
- **reserved**
  Reserved.
- **content**
  The specific content of a message.

3.5.12 InnoXyzPoint

The information of points in the 3D coordinate axis.

Syntax

```
/* compact format, 16 + 8 = 24 bytes per point */
struct InnoXyzPoint {
  float x;
  float y;
  float z;
  float radius;
  uint16_t ts_10us;
  uint16_t scan_id: 9;   /* id of the scan line */
  uint16_t in_roi: 2;
  uint16_t facet: 3;
  uint16_t reserved_flags: 2; /* all 0 */
  uint32_t is_2nd_return: 1;
  uint32_t scan_idx: 14;   /* point idx within the scan line */
  uint32_t refl: 9;        /* reflectance, 1-254, 255 means a reflector     */
                           /* or intensity, also 1-254 & 255=reflector      */
  uint32_t type: 2;        /* 0: normal, 1: ground, 2: fog                  */
  uint32_t elongation: 4;  /* elongation */
  uint32_t channel: 2;
};
```

Members

- **x**
  The x-axis value. The unit is meter.
- **y**
  The y-axis value. The unit is meter.
- **z**
  The z-axis value. The unit is meter.
- **radius**
  The distance from the origin point. The unit is 0.5 centimeters.
- **ts_10us**
  Time relative to the laser pulse. The unit is 10 us.

- **scan_id**
  Scanning line ID.
- **in_roi**
  The position of the point is relative to the ROI.
  in_roi=0x00: in sparse region
  in_roi=0x11: in center of ROI
- **facet**
  The polygon mirror serial number where the point is located. (There are 5 mirrors, and the serial number is 0 to 4)
- **reserved_flags**
  Reserved
- **is_2nd_return**
  is_2nd_retun=1 means the dual return mode. is_2nd_retun=0 means the single return mode.
- **scan_idx**
  The ID of the point in the scanning line.
- **refl**
  Reflectance. The range is 1 to 254. The refl=255 means total reflectance.
- **type**
  The reflectance type.
- **elongation**
  The pulse width.
- **channel**
  The channel serial number where the point is located.

Copyright©Innovusion

4 innovusion_lidar_util Reference

The innovusion_lidar_util commands mainly provide the functions to change and get LiDAR-related parameters.

The steps of innovusion_lidar_util usage are as follows.

1. Unzip the SDK tgz file to inno_sdk.

   Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Execute the command.

```
./innovusion_lidar_util <parameters>        // Run the innovusion_lidar_util tool
```

4.1 get_version

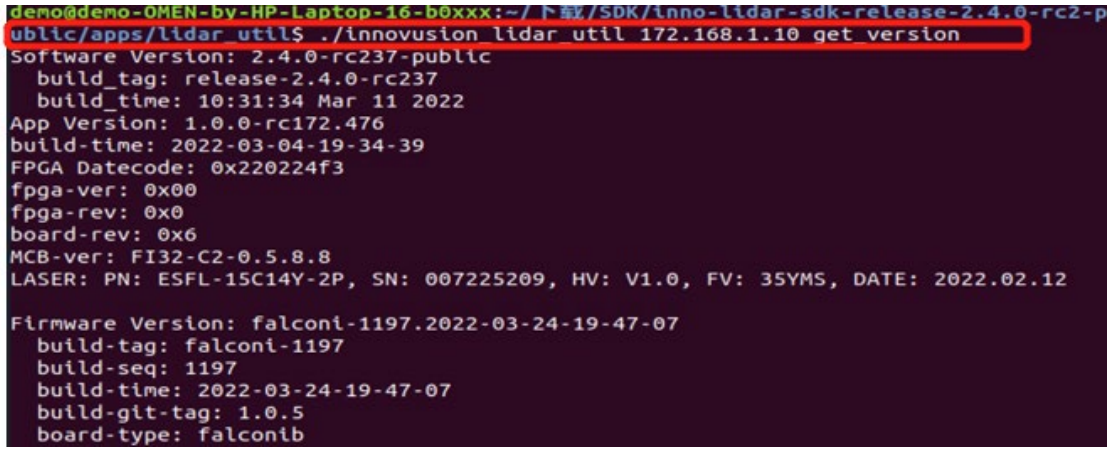
Get detailed SW/FW version information of the LiDAR.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `./innovusion_lidar_util <ip of LIDAR> get_version` to get detailed version information of the SW/FW.

   `<ip of LIDAR>`—LiDAR IP



4.2 start <file_name> <capture_size_in_MB>

Download data.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `./innovusion_lidar_util <ip of LIDAR> start <file_name> <capture_size_in_MB>` to capture and download the raw data. `capture_size_in_MB` in the result must be greater than zero.

- `<ip of LIDAR>`—LiDAR IP
- `<file_name>`—Name of the downloaded data file
- `<capture_size_in_MB>`—Unit in MB (if it is not a positive number, it will be indefinitely streamed, but no data will be saved)

```
$ ./innovusion_lidar_util 10.42.0.91 start ./rawdata 12
capture_size: 12MB
Stopped.
id: 1
Stopped.
data recorded in ./rawdata_ADCData_02_25_2022_22_34_51.dat
```

4.3 get_error_code

Get the error code.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run ./innovusion_lidar_util <ip of LIDAR> get_error_code to get the error code.

   `<ip of LIDAR>`—LiDAR IP

```
$ ./innovusion_lidar_util 10.42.0.91 get_error_code
Error Code: 0x0
```

4.4 set_ip_address

Change the LiDAR IP. Currently, only the IP address in the same network segment can be changed.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `./innovusion_lidar_util <ip of LIDAR> set_ip_address <new_ip_address>.`
3. Reboot the LiDAR with the soft reboot command（`./innovusion_lidar_util <old ip address> soft_reboot`）or the hard reboot (power reboot).
4. (Optional) You can check if the IP has been changed by viewing the LiDAR network information or logging into the ILA.

- `<ip of LIDAR>`—LiDAR IP
- `<new_ip_address>`—New LiDAR IP

4.5 set_network

Change the LiDAR network information.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `./innovusion_lidar_util <ip of LIDAR> set_network <new_ip_address> <new_netmask_address> <new_gateway_address>`。
3. Reboot the LiDAR with the soft reboot command（`./innovusion_lidar_util <old ip address> soft_reboot`）or the hard reboot (power reboot).
4. (Optional) You can check if the IP has been changed by viewing the LiDAR network information or logging into the ILA.

- `<ip of LIDAR>`—LiDAR IP
- `<new_ip_address>`—New LiDAR IP
- `<new_netmask_address>`—New subnet mask
- `<new_gateway_address>`—New gateway

Copyright©Innovusion

```
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc237-public/apps/lidar_util$ ./innovusion_lidar_util 172.168.1.10 get_network
IP=172.168.1.10 netmask=255.255.255.0 gateway=172.168.1.1
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc237-public/apps/lidar_util$ ./innovusion_lidar_util 172.168.1.10 set_network 172.168.1.10 255.255.0.0 172.168.1.1
Set network:  IP=172.168.1.10 netmask=255.255.0.0 gateway=172.168.1.1
Please power cycle the LIDAR to make changes effective.
```

4.6 get_network

View the LiDAR network information.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `./innovusion_lidar_util <ip of LIDAR> get_network`.

   `<ip of LIDAR>`—LiDAR IP

```
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc237-public/apps/lidar_util$ ./inno
vusion_lidar_util 172.168.1.10 get_network
IP=172.168.1.10 netmask=255.255.0.0 gateway=172.168.1.1
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc237-public/apps/lidar_util$
```

4.7 get_uptime

View the total uptime of LiDAR.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `./innovusion_lidar_util <ip of LIDAR> get_uptime`.

   `<ip of LIDAR>`—LiDAR IP

```
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc2
-public/apps/lidar_util$ ./innovusion_lidar_util 172.168.1.10 get_uptime
Sensor lifelong uptime: 74.5 hours
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc2
-public/apps/lidar_util$
```

4.8 get_framerate

Get the LiDAR frame rate information.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `./innovusion_lidar_util <ip of LIDAR> get_framerate`.

   `<ip of LIDAR>`—LiDAR IP

```
-public/apps/lidar_util$ ./innovusion_lidar_util 172.168.1.10 get_framerate
6.00000
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc2
-public/apps/lidar_util$
```

4.9 soft_reboot

Soft reboot.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `./innovusion_lidar_util <ip of LIDAR> soft_reboot`。

3. (Optional) You can check if the LiDAR pointcloud is normal via the ILA or the ping command.

   `<ip of LIDAR>`—LiDAR IP

```
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc2
-public/apps/lidar_util$ ./innovusion_lidar_util 172.168.1.10 soft_reboot
reboot...
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc2
-public/apps/lidar_util$ ping 172.168.1.10
PING 172.168.1.10 (172.168.1.10) 56(84) bytes of data.
^C
--- 172.168.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3065ms

demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc2
-public/apps/lidar_util$ ping 172.168.1.10
PING 172.168.1.10 (172.168.1.10) 56(84) bytes of data.
64 bytes from 172.168.1.10: icmp_seq=1 ttl=64 time=0.211 ms
64 bytes from 172.168.1.10: icmp_seq=2 ttl=64 time=0.265 ms
```

4.10 get_temperature

Get the temperature information.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `./innovusion_lidar_util <ip of LIDAR> get_temperature`.

   `<ip of LIDAR>`—LiDAR IP

```
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc237-public/apps/lidar_util$ ./inno
vusion_lidar_util 172.168.1.10 get_temperature
temps: T0=60,T1=61,T2=59,Tlaser=59,Txadc=60
```

4.11 get_log

View all firmware log information.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `./innovusion_lidar_util <ip of LIDAR> get_log` to get the log information.

   `<ip of LIDAR>`—LiDAR IP

Copyright©Innovusion

```
2021-12-20 00:15:11.688 [ INFO] dsp events: 0x10
2021-12-20 00:15:11.688 [ERROR] s...h.cc:1006 FAULT set: 15
2021-12-20 00:15:11.692 [ INFO] MCB TX: STW10800016ND
2021-12-20 00:15:11.794 [ INFO] MCB RX: AC
2021-12-20 00:15:11.795 [ INFO] dsp event interrupt
2021-12-20 00:15:11.795 [ INFO] MCB TX: STR108ND
2021-12-20 00:15:11.897 [ INFO] MCB RX: 00000
2021-12-20 00:15:11.897 [ INFO] dsp events: 0x0
2021-12-20 00:15:11.898 [ INFO] MCB TX: STR108ND
2021-12-20 00:15:12.000 [ INFO] MCB RX: 00016
2021-12-20 00:15:12.000 [ INFO] dsp events: 0x10
2021-12-20 00:15:12.000 [ERROR] s...h.cc:1006 FAULT set: 15
2021-12-20 00:15:12.004 [ INFO] MCB TX: STW10800016ND
2021-12-20 00:15:12.106 [ INFO] MCB RX: AC
2021-12-20 00:15:12.107 [ INFO] dsp event interrupt
2021-12-20 00:15:12.107 [ INFO] MCB TX: STR108ND
2021-12-20 00:15:12.209 [ INFO] MCB RX: 00000
2021-12-20 00:15:12.209 [ INFO] dsp events: 0x0
2021-12-20 00:15:12.210 [ INFO] MCB TX: STR108ND
2021-12-20 00:15:12.312 [ INFO] MCB RX: 00016
2021-12-20 00:15:12.313 [ INFO] dsp events: 0x10
```

4.12 get_log_file

Download the LiDAR's firmware log to a local disk.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `./innovusion_lidar_util <ip of LIDAR> get_log_file <filename>` to download the LiDAR's firmware log to a local directory.

- `<ip of LIDAR>`—LiDAR IP
- `<filename>`—Name of the log file.

```
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc237-public/apps/lidar_util$  ./inn
ovusion_lidar_util 172.168.1.10 get_log_file log
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc237-public/apps/lidar_util$ ll
总用量 26796
drwxrwxr-x 3 demo demo     4096 3月  25 18:42 ./
drwxrwxr-x 6 demo demo     4096 3月  25 14:25 ../
-rwxrwxr-x 1 demo demo  1141256 3月  11 18:31 innovusion_lidar_util*
-rw-r--r-- 1 demo demo 23392625 3月  25 18:42 log
drwxrwxr-x 4 demo demo     4096 3月  25 14:25 scripts/
-rwxrwxr-x 1 demo demo     1153 3月  11 18:29 split_yaml_raw.bash*
-rw-r--r-- 1 demo demo  2883584 3月  25 17:43 test01_ADCData_03_25_2022_17_43_19.dat
```

4.13 get_sn

Get the LiDAR's SN number.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `./innovusion_lidar_util <ip of LIDAR> get_sn`.

   `<ip of LIDAR>`—LiDAR IP

```
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc237-public/apps/lidar_util$ ./inno
vusion_lidar_util 172.168.1.10 get_sn
serial number: 370602210028
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc237-public/apps/lidar_util$
```

4.14 get_status

Get the LiDAR's status information.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `./innovusion_lidar_util <ip of LIDAR> get_status`.

   `<ip of LIDAR>`—LiDAR IP

```
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc237-public/apps/lidar_util$ ./inno
vusion_lidar_util 172.168.1.10 get_status
time_config=1 up_time=31392501967504 err=0x0 stream_status=2 stream_count=4 data_sent=1835008 count1=129 count2=0
 count3=0
```

4.15 get_model

Get the LiDAR's model information.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `./innovusion_lidar_util <ip of LIDAR> get_model`.

   `<ip of LIDAR>`—LiDAR IP

```
demo@demo-OMEN-by-HP-Laptop-16-b0xxx:~/下载/SDK/inno-lidar-sdk-release-2.4.0-rc237-public/apps/lidar_util$  ./inn
ovusion_lidar_util 172.168.1.10 get_model
model: k
```

4.16 get_config_changed

View the LiDAR's parameters that were changed.

1. Enter the lidar_util path.

```
cd /apps/lidar_util
```

2. Run `innovusion_lidar_util <ip of LIDAR>get_config_changed <section> <key>…`.

- `<section>`—manufacturer or time
- `<key>`—Parameter name
- `<value>`—Parameter value

```
$ ./innovusion_lidar_util 172.168.1.10 get_config_changed
```

```
(base) demo@INNO-SZV00173:/mnt/d/FalconK及AI测试/SDK/2.4.0-rc237-pub
[manufacture]
internal_server = 1
```


Copyright©Innovusion

Table 5 Revision History

Version	Date	Remarks
V1.0	20221110	The draft, including the API instruction, usage of SDK and lidar_util